Data Structure Algorithm & Application (CT-159) Lab 01

Dynamic Arrays, Linear, and Binary Search

Objectives

The objective of this experiment to get familiar with

1. Arrays and its role in data structure
2. 2D Dynamic Rectangular Array and Jagged Array
3. Linear and binary searching techniques

Tools Required

Dev C++ IDE

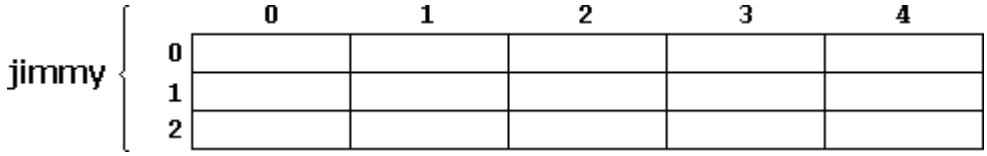
Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

Introduction

An array is a series of elements of the same data type placed in contiguous memory locations that can be individually referenced by adding an index to a unique identifier.

int arr[5]

For example, five values of type **int** can be declared as an array without having to declare 5 differentvariables (each with its own identifier). These values can be accessed using the same identifier, with the proper index. **Multidimensional arrays** can be described as "arrays of arrays". For example, a bi-dimensional array can be imagined as a two-dimensional table made of elements, all of them of a same uniform data type.



Jimmy represents a bi-dimensional array of 3 per 5 elements of type int as shown in above Figure. The C++ syntax for this is: **int jimmy[3][5].** In addition, for example, the way to reference the second element vertically and fourth horizontally in anexpression would be: **jimmy[1][3].**

Dynamic Memory Allocation for arrays:

Memory in your C++ program is divided into two parts

1. The **stack –** All variables declared inside the function will take up memory from the stack.
2. The **heap –** this is unused memory and can be used to allocate the memory dynamically during program execution.

A **dynamic array** is an array with a big improvement, that is, **automatic resizing**. One limitation of static arrays is that they're fixed size, meaning you need to specify the number of elements your array will hold ahead of time. A dynamic array expands as you add more elements. So you don't need to determine the size ahead of time.

Strengths:

1. **Fast lookups**. Just like static arrays, retrieving the element at a given index takes O(1) time.
2. **Variable size.** You can add as many items as you want, and the dynamic array will expand to hold them.
3. **Cache-friendly.** Just like static arrays, dynamic arrays place items right next to each other in memory, making efficient use of caches.

Weaknesses:

1. **Slow worst-case appends.** Usually, adding a new element at the end of the dynamic array takes **O(1)** time. But if the dynamic array doesn't have any room for the new item, it'll need to expand, which takes **O(n)** time.
2. **Costly inserts and deletes.** Just like static arrays, elements are stored adjacent to each other. So adding or removing an item in the middle of the array requires "scooting over" other elements, which takes **O(n)** time.

Factors impacting the performance of Dynamic Arrays:

The array's initial size and its growth factor determine its performance. Note the following points:

1. If an array has a **small size** and a **small growth factor**, it will keep on **reallocating** memory more often. This will **reduce** the performance of the array.
2. If an array has a **large size** and a **large growth factor**, it will have a **huge chunk** of **unused** memory. Due to this, resize operations may take longer. This will reduce the performance of the array.

The new Keyword:

In C++, we can create a dynamic array using the **new keyword**. The number of items to be allocated is specified within a pair of square brackets. The type name should precede this. The requested number of items will be allocated.

```
int *ptr1 = new int;
int *ptr1 = new int[5];
int *array { new int[10]{}};
int *array { new int[10]{1,2,3,4,5,6,7,8,9,10}};
```

Resizing Arrays:

The length of a dynamic array is set during the allocation time.However, C++ doesn't have a built-in mechanism for resizing an array once it has been allocated. You can, however, overcome this challenge by allocating a new array dynamically, copying over the elements, then erasing the old array.

Dynamically Deleting Arrays:

A dynamic array should be deleted from the computer memory once its purpose is fulfilled. The delete statement can help you accomplish this. The released memory space can then be used to hold another set of data. However, even if you do not delete the dynamic array from the computer memory, it will be deleted automatically once the program terminates.

```
delete ptr;
delete[] array;
```

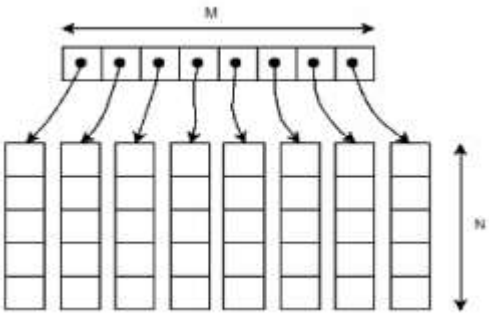
NOTE: To delete a dynamic array from the computer memory, you should use delete[], instead of delete. The [] instructs the CPU to delete multiple variables rather than one variable. The use of delete instead of delete[] when dealing with a dynamic array may result in problems. Examples of such problems include **memory leaks, data corruption, crashes,** etc.

Example 01: Single Dimensional Array memory allocation

```
#define N 10
int main(){
        // dynamically allocate memory of size 5 and assign values to allocated memory
       int *array{ new int[5]{ 10, 7, 15, 3, 11 } };
        // print the 1D array de allocate memory
}
```

Two Dimensional Array Using Array of Pointers:

We can dynamically create an array of pointers of size M and then dynamically allocate memory of size N for each row as shown below.



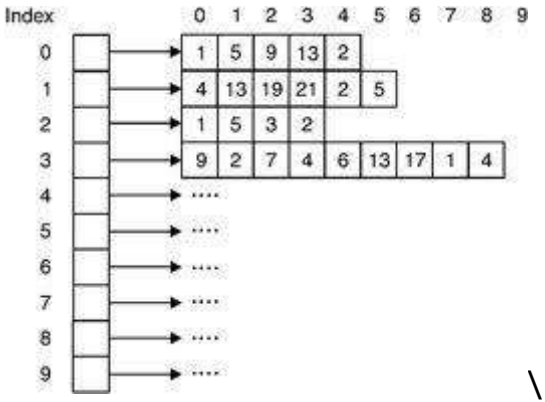
Example 02: 2D dynamic array memory allocation

```
// Dynamically Allocate Memory for 2D Array in C++
int main(){
```

```
//Enter two dimensions N and M, Dynamically allocate memory to both
int** ary = new int*[N];
 for(int i = 0; i < N; ++i)
    ary[i] = new int[M];
//fill the arrays, print them, deallocate the memory
}
```

Jagged Array

Jagged array is nothing but it is **an array of arrays** in which the member arrays can be in different sizes.



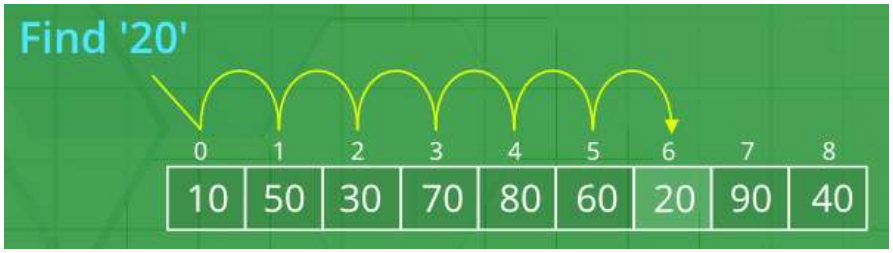
\

Example 03: Jagged Array

```
int **arr = new int*[3];
int Size[3], i,j,k;
for(i=0;i<3;i++){
        cout<<"Row "<<i+1<< " size: ";
        cin>>Size[i];
        arr[i] =new int[Size[i]]; }
for(i=0;i<3;i++){
        for(j=0;j<Size[i];j++){
                cout<<"Enter row " <<i+1<<" elements: ";
                cin>>*(*(arr + i) + j);  }
}
```

Linear Search:

Linear Search is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set. It is the easiest searching algorithm as shown in the following figure.



Algorithm:

1. Start from the leftmost element of arr[] and one by one compare x with each element of arr[],
2. If x matches with an element, return the index,
3. If x doesn't match with any of the elements, return -1

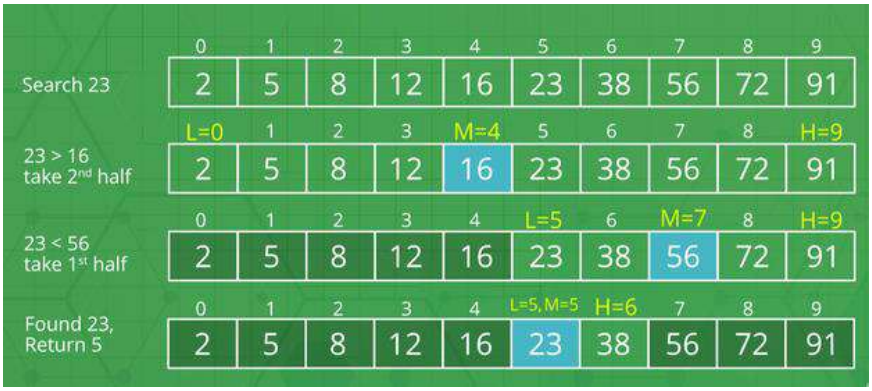
Example 04: Linear Search

```
#include <iostream>
```

```
using namespace std;
int search(int arr[], int N, int x){
   int i;
   for (i = 0; i < N; i++)
      if (arr[i] == x)
         return i;
   return -1;
}
int main(void){
   int arr[] = { 2, 3, 4, 10, 40 };
   int x = 10;
   int N = sizeof(arr) / sizeof(arr[0]);
   // Function call
   int result = search(arr, N, x);
   (result == -1)? cout << "Element is not present in array": cout << "Element is present at index "  <<
result;
   return 0;}
```

Binary Search:

Binary Search is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(Log n) as shown in Figure 1.5.



Algorithm:

1. Compare x with the middle element.
2. If x matches with the middle element, we return the mid index.
3. Else If x is greater than the mid element, then x can only lie in the right half subarray after the mid element. So we recur for the right half.
4. Else (x is smaller) recur for the left half.

Example 05: Binary Search (Iterative Approach)

```
using namespace std;
// A iterative binary search function. It returns
// location of x in given array arr[l..r] if present, otherwise -1
int binarySearch(int arr[], int l, int r, int x){
   while (l <= r) {
      int m = l + (r - l) / 2;
      // Check if x is present at mid
      if (arr[m] == x)
         return m;
      // If x greater, ignore left half
      if (arr[m] < x)
         l = m + 1;
      // If x is smaller, ignore right half
      else
         r = m - 1;
   }
```

```
   // if we reach here, then element was
   // not present
   return -1;
}
int main( ){
   int arr[ ] = { 2, 3, 4, 10, 40 };
   int x = 10;
   int n = sizeof(arr) / sizeof(arr[0]);
   int result = binarySearch(arr, 0, n - 1, x);
   (result == -1) ? cout << "Element is not present in array": cout << "Element is present at index "
<< result;
   return 0;
}
```

Exercise

1. Write a C++ program to copy data of a 2D array in a 1D array using Column Major Order.
2. Write a program to calculate the GPA of students of all subjects of a single semester . Assume all the courses have the same credit hour (let's assume 3 credit hours).

	Data Structure	**Programming for AI**	**Digital Logic Design**	**Probability & Statistics**	**Finance & Accounting**
Ali	3.66	3.33	4.0	3.0	2.66
Hiba	3.33	3.0	3.66	3.0	---
Asma	4.0	3.66	2.66	---	---
Zain	2.66	2.33	4.0	---	---
Faisal	3.33	3.66	4.0	3.0	3.33

3. The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.
   For example, for arr = [2,3,4], the median is 3.
   For example, for arr = [2,3], the median is (2 + 3) / 2 = 2.5.
   Implement the MedianFinder class:
   - MedianFinder() initializes the MedianFinder object.
   - void addNum(int num) adds the integer num from the data stream to the data structure.
   - double findMedian() returns the median of all elements so far. Answers within 10-5 of the actual answer will be accepted.

   Example 1:
   Input: ["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
   [[], [1], [2], [], [3], []]
   Output: [null, null, null, 1.5, null, 2.0]
   Explanation
   MedianFinder medianFinder = new MedianFinder();
   medianFinder.addNum(1); // arr = [1]
   medianFinder.addNum(2); // arr = [1, 2]
   medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
   medianFinder.addNum(3); // arr[1, 2, 3]
   medianFinder.findMedian(); // return 2.0
   Constraints: -105 <= num <= 105
   There will be at least one element in the data structure before calling findMedian.
   At most 5 * 104 calls will be made to addNum and findMedian.
4. Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1. You must write an algorithm with O(log n) runtime complexity.
   Example 1: Input: nums = [-1,0,3,5,9,12], target = 9, Output: 4
   Explanation: 9 exists in nums and its index is 4
   Example 2: Input: nums = [-1,0,3,5,9,12], target = 2, Output: -1
   Explanation: 2 does not exist in nums so return -1
   Constraints:
   1 <= nums.length <= 104

-104 < nums[i], target < 104
All the integers in nums are unique.

5. nums is sorted in ascending order.You are given an m x n integer matrix with the following two properties: Each row is sorted in non-decreasing order. The first integer of each row is greater than the last integer of the previous row. Given an integer target, return true if target is in matrix or false otherwise. You must write a solution in O(log(m * n)) time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3, Output: true
Constraints:
m == matrix.length, n == matrix[i].length, 1 <= m, n <= 100, -104 <= matrix[i][j], target <= 104

Lab 01 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		

Data Structure Algorithm & Application (CT-159) Lab 02

Recursion, Backtracking, Stack

Objectives

The objective of the lab is to get students familiar with recursion and stack, and their usage in backtracking Algorithm.

Tools Required

Dev C++ IDE

Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

Recursion - Introduction

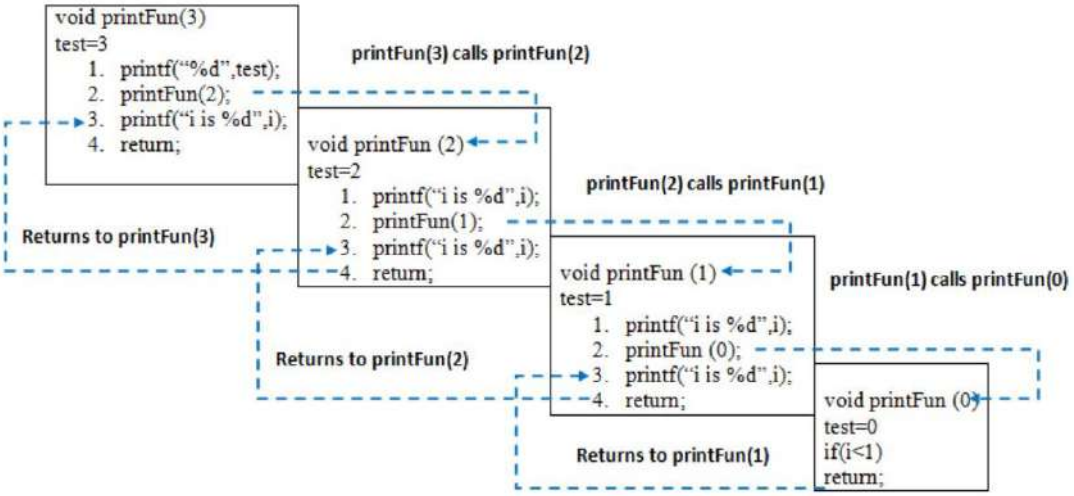
The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function. Using a recursive algorithm, certain problems can be solved quite easily. Examples of such problems are Towers of Hanoi (TOH), Inorder/Preorder/Postorder Tree Traversals, DFS of Graph, etc. A recursive function solves a particular problem by calling a copy of itself and solving smaller subproblems of the original problems.

Properties of Recursion:

- Performing the same operations multiple times with different inputs.
- In every step, we try smaller inputs to make the problem smaller.
- Base condition is needed to stop the recursion otherwise infinite loop will occur.

Example 01:

```
// A C++ program to demonstrate working of  recursion
#include <iostream>
using namespace std;

void printFun(int test){
        if (test < 1)
                return;
        else {
                cout << test << " ";
                printFun(test - 1); // statement 2
                cout << test << " ";
                return;
        }
}
// Driver Code
int main(){
        int test = 3;
        printFun(test);
}
```



Tail and Non-Tail Recursion

Tail recursion is a specific type of recursion where the recursive call is the last operation in the function. This means that after the recursive call, the function does nothing else; it returns the result of the recursive call directly.

Example 02: Find the greatest common divisor (GCD) using tail recursion.

```
#include <iostream>
using namespace std;
// Tail-recursive GCD function
int gcdTailRecursive(int a, int b) {
   if (b == 0) {
```

```
    return a;  // Base case
  } else {
    return gcdTailRecursive(b, a % b);  // Tail-recursive call
  }
}
int main() {
  int a = 56;
  int b = 98;
  int result = gcdTailRecursive(a, b);
  cout << "GCD of " << a << " and " << b << " is: " << result << endl;
  return 0;
}
```

Non-tail recursion refers to a recursive function where the recursive call is **not** the last operation performed in the function. In other words, after the recursive call, the function still has to perform some additional computations or operations before returning a result. Because the recursive call is not the final action, the function cannot immediately return the result of the recursive call. Instead, it has to keep the current function's state on the stack until all recursive calls complete. This often leads to increased memory usage and can potentially result in stack overflow for deep recursion.

Example 03: Find Fibonacci series till term n.

```
int fibonacci(int n) {
  if (n <= 1) return n; // Base case
  return fibonacci(n - 1) + fibonacci(n - 2); // Recursive case
}
```

```
                    fibonacci(4)
                   /            \
            fibonacci(3)        fibonacci(2)
            /        \          /        \
     fibonacci(2)  fibonacci(1)  fibonacci(1)  fibonacci(0)
       /   \
fibonacci(1) fibonacci(0)
```

Backtracking – Introduction

In solving some problems, a situation arises where there are different ways leading from a given position, none of them known to lead to a solution. After trying one path unsuccessfully, we return to this crossroads and try to find a solution using another path. However, we must ensure that such a return is possible and that all paths can be tried. This technique is called *backtracking*, and it allows us to systematically try all available avenues from a certain point after some of them lead to nowhere. Using backtracking, we can always return to a position that offers other possibilities for successfully solving the problem. This technique is used in artificial intelligence, and one of the problems in which backtracking is very useful is the eight queens problem.

How many Subsets are possible for an Array of size 'N' ?

Number of Subsets of an array of size N = 2^N

Proof: For each element of the array we have 2 choices:

- **Choice 1:** Include it into the subset.
- **Choice 2:** Exclude it from the subset.

Since, each element has 2 choice to contribute into the subset and we have total N elements, therefore total subsets = 2^N. Backtracking algorithm can allow us to explore all possible choices one by one recursively. It uses non-tail recursion.

Backtracking Algorithm:

1. **Initialize**: Start with an empty subset and an empty list to store all subsets.
2. **Recursive Function**: Define a recursive function that takes the current index and the current subset as parameters.
   - **Base Case**: If the current index equals the size of the input set, add the current subset to the list of all subsets.
   - **Recursive Case**:

- **Include the element**: Add the current element to the subset.
- **Explore the rest of the elements:** by recursively calling the function for the next index.
- **Exclude the element (Backtrack)**: After exploring all possibilities for the current element, backtrack by removing the last added element from the current subset to explore other combinations.

Example 04: Find all possible subsets of a set with N elements.

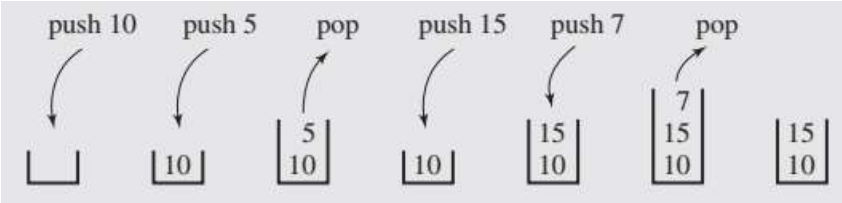
```
#include <iostream>
#include <vector>
using namespace std;

void calcSubset(vector<int>& A, vector<vector<int> >& res, vector<int>& subset, int index){
   // Add the current subset to the result list
   res.push_back(subset);
   // Generate subsets by recursively including and excluding elements
   for (int i = index; i < A.size(); i++) {
      // Include the current element in the subset
      subset.push_back(A[i]);

      // Recursively generate subsets with the current element included
      calcSubset(A, res, subset, i + 1);

      // Exclude the current element from the subset (backtracking)
      subset.pop_back();
   }
}

vector<vector<int> > subsets(vector<int>& A) {
   vector<int> subset;
   vector<vector<int> > res;
   int index = 0;
   calcSubset(A, res, subset, index);
   return res;
}

// Driver code
int main(){
   vector<int> array = { 1, 2, 3 };
   vector<vector<int> > res = subsets(array);

   // Print the generated subsets
   for (int i = 0; i < res.size(); i++) {
      for (int j = 0; j < res[i].size(); j++)
         cout << res[i][j] << " ";
      cout << endl;
   }

   return 0;
}
```

Stack - Introduction

A stack is a linear data structure that can be accessed only at one of its ends for storing and retrieving data. Such a stack resembles a stack of trays in a cafeteria: new trays are put on the top of the stack and taken off the top. The last tray put on the stack is the first tray removed from the stack. For this reason, a stack is called an LIFO structure: last in/first out.

A stack is defined in terms of operations that change its status and operations that check this status. The operations are as follows:

- ✓ clear()—Clear the stack.
- ✓ isEmpty()—Check to see if the stack is empty.
- ✓ push(el)—Put the element el on the top of the stack.
- ✓ pop()—Take the topmost element from the stack.
- ✓ topEl()—Return the topmost element in the stack without removing it.



Example 03: Implementation of Stack using array.

```
#include <iostream>
using namespace std;
class Stack {
private:
   int* arr;       // Pointer to the array that holds the stack elements
   int capacity;    // Maximum size of the stack
   int topIndex;    // Index of the top element in the stack
public:
   // Constructor to initialize the stack with a given capacity
   Stack(int size) {
      capacity = size;
      arr = new int[capacity];
      topIndex = -1;  // Indicates an empty stack
   }
   // Destructor to clean up the allocated memory
   ~Stack() {
      delete[] arr;
   }

   // Push an element onto the stack
   void push(int element) {
      if (topIndex == capacity - 1) {
         cout << "Stack overflow. Unable to push " << element << endl;
         return;
      }
      arr[++topIndex] = element;
   }

   // Pop an element from the stack
   int pop() {
      if (topIndex == -1) {
         cout << "Stack underflow. Unable to pop." << endl;
         return -1;  // Returning -1 to indicate an error (assuming only positive numbers are used)
      }
      return arr[topIndex--];
   }
   // Peek at the top element of the stack
   int peek() const {
      if (topIndex == -1) {
         cout << "Stack is empty." << endl;
         return -1;  // Returning -1 to indicate an error
      }
      return arr[topIndex];
   }
```

```
    // Check if the stack is empty
    bool isEmpty() const {
        return topIndex == -1;
    }
    // Get the current size of the stack
    int size() const {
        return topIndex + 1;
    }
};
int main() {
    Stack stack(5);  // Create a stack with a capacity of 5
    stack.push(10);
    stack.push(20);
    stack.push(30);
    cout << "Top element is: " << stack.peek() << endl;
    cout << "Stack size is: " << stack.size() << endl;
    stack.pop();
    cout << "Top element after pop is: " << stack.peek() << endl;
    cout << "Stack size after pop is: " << stack.size() << endl;
    return 0;
}
```

Applications

1. Delimiter Matching

One application of the stack is in matching delimiters in a program. This is an important example because delimiter matching is part of any compiler: No program is considered correct if the delimiters are mismatched. In C++ programs, we have the following delimiters: parentheses “(” and “)”, square brackets “[” and “]”, curly brackets “{” and “}”, and comment delimiters “/*” and “ */”.

The delimiter matching algorithm reads a character from a C++ program and stores it on a stack if it is an opening delimiter. If a closing delimiter is found, the delimiter is compared to a delimiter popped off the stack. If they match, processing continues; if not, processing discontinues by signaling an error. The processing of the C++ program ends successfully after the end of the program is reached and the stack is empty.

Pseudocode

```
function isValidDelimiterString(s):
   stack = empty stack
   for each character in s:
      if character is one of '(', '{', '[':
         push character onto stack
      else if character is one of ')', '}', ']':
         if stack is empty:
            return False
         top = pop from stack
         if character does not match the type of top:
            return False
   return stack is empty
```

2. Postfix Expression

A postfix expression is an expression in which each operator follows its operands. The advantage of this form is that there is no need to group sub-expressions in parentheses or to consider operator precedence. Stack can be used to evaluate a postfix (or prefix) expression.

Pseudocode

```
function evaluatePostfix(expression):
  stack = empty stack

  for each token in expression:
    if token is an operand:
      Push token onto the stack
    else if token is an operator:
      Operand2 = Pop from stack
      Operand1 = Pop from stack
      Result = Apply operator on Operand1 and Operand2
      Push Result onto the stack

  // After processing all tokens, the result is the only element left in the stack
  return Pop from stack
```

3. Tower of Hanoi

The **Tower of Hanoi** is a classic problem in computer science and mathematics that involves moving a set of disks from one peg (or rod) to another, following specific rules. It is an excellent example for teaching the concept of recursion and backtracking due to its simple rules and recursive solution.

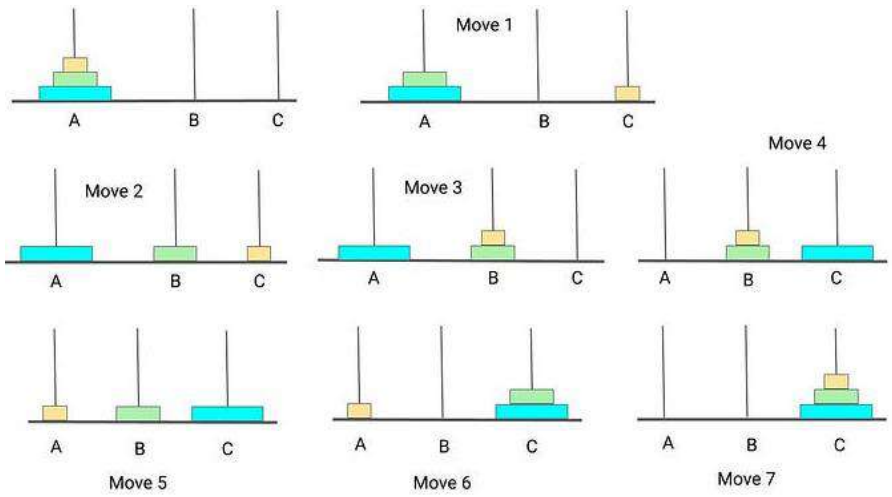
Problem Statement

You have three rods (usually named A, B, and C) and a number of disks of different sizes. Initially, all disks are stacked in increasing size on one rod, with the largest disk at the bottom and the smallest disk at the top.

The goal is to move the entire stack of disks from the initial rod (let's say rod A) to another rod (let's say rod C), using a third rod (rod B) as an auxiliary.

Rules

I. Only one disk can be moved at a time.
II. A disk can only be placed on top of a larger disk or on an empty rod.
III. All disks must be moved from the source rod to the destination rod.



Example: Consider 3 disks labelled 1, 2, and 3 in increasing order of size, with 1 being the smallest and 3 being the largest:

- **Initial State**: All disks are on rod A.
- **Goal**: Move all disks to rod C, using rod B as an auxiliary rod.

Recursive Algorithm for Tower of Hanoi

The problem can be solved recursively by breaking it down into smaller subproblems:

I. **Base Case**: If there is only one disk, move it directly from the source rod 'A' to the destination rod 'C'.
II. **Recursive Case**: If there are n disks:

- **Move `n-1` disks** from rod `A` to rod `B` using rod `C` as an auxiliary.
- **Move the nth disk** (largest disk) from rod `A` to rod `C`.
- **Move `n-1` disks** from rod `B` to rod `C` using rod `A` as an auxiliary.

```
void towerOfHanoi(int n, char sourceRod, char destinationRod, char auxiliaryRod) {
  if (n == 1) {
    // Base case: only one disk, move it from source to destination
    cout << "Move disk 1 from " << sourceRod << " to " << destinationRod << endl;
    return;
  }

  // Step 1: Move n-1 disks from source to auxiliary rod using destination as auxiliary
  towerOfHanoi(n - 1, sourceRod, auxiliaryRod, destinationRod);

  // Step 2: Move the nth disk from source to destination
  cout << "Move disk " << n << " from " << sourceRod << " to " << destinationRod <<endl;

  // Step 3: Move n-1 disks from auxiliary to destination rod using source as auxiliary
  towerOfHanoi(n - 1, auxiliaryRod, destinationRod, sourceRod);
}
```

4. Finding a path in a Maze

Maze solving using backtracking is another classic problem where we determine if there is a path from a starting point to an end point in a maze. The maze is typically represented by a 2D grid where some cells are open and others are blocked.

Problem Statement

Given a maze represented as a 2D grid of cells, where:

- 1 represents an open path.
- 0 represents a blocked path.

The task is to determine if there is a path from a starting cell (usually the top-left corner (0, 0)) to an ending cell (usually the bottom-right corner (n-1, m-1)) using only valid moves (typically up, down, left, or right).

Algorithm for Maze Solving Using Backtracking

The maze-solving problem can be solved using a backtracking approach. The basic idea is to start from the starting cell and explore all possible paths using depth-first search (DFS). If a path leads to the end, the maze is solvable; otherwise, we backtrack and try a different path.

Algorithm Steps

I. **Base Case**:
   - If the current cell is the destination (end point), return true.
   - If the current cell is blocked or out of bounds, return false.

II. **Recursive Case**:
   - Mark the current cell as part of the path (visited).
   - Recursively try moving in all four possible directions (up, down, left, right).
   - If moving in any direction returns true, the path is found.
   - If all directions return false, backtrack by unmarking the current cell and return false.

Pseudocode

```
function solveMaze(maze, x, y, solution):
  if (x, y) is the destination:
    mark solution[x][y] as part of the path
    return True

  if (x, y) is a valid cell and not visited:
    mark solution[x][y] as part of the path
```

```
    // Move right
    if solveMaze(maze, x, y + 1, solution):
        return True

    // Move down
    if solveMaze(maze, x + 1, y, solution):
        return True

    // Move left
    if solveMaze(maze, x, y - 1, solution):
        return True

    // Move up
    if solveMaze(maze, x - 1, y, solution):
        return True

    // Backtrack: unmark (x, y) as part of the path
    unmark solution[x][y]
    return False

return False
```

Exercise

1. A palindrome is a word, phrase, number, or another sequence of characters that reads the same backward and forwards. Can you determine if a given string, s, is a palindrome? Write a Program using stack for checking whether a string is palindrome or not.
2. Given two strings s and t, return true if they are equal when both are typed into empty text editors. '#' means a backspace character. Note that after backspacing an empty text, the text will continue empty.
   Example 1: Input: s = "ab#c", t = "ad#c", Output: true, Explanation: Both s and t become "ac".
   Example 2: Input: s = "a#c", t = "b", Output: false, Explanation: s becomes "c" while t becomes "b".
3. Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order.
   Example 1: Input: nums = [1,2,3], Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
   Example 2: Input: nums = [0,1], Output: [[0,1],[1,0]]
4. Given an m x n grid of characters board and a string word, return true *if* word *exists in the grid*. The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.
   Example 01: Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCCED", **Output:** true

A	B	C	E
S	F	C	S
A	D	E	E

   Example 02: Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCB", **Output:** false

A	B	C	E
S	F	C	S
A	D	E	E

Lab 01 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		

Data Structure Algorithms & Applications (CT-159)
Lab 03

Queue, Circular Queue, Double Ended Queue

Objectives

The objective of the lab is to get students familiar with Queue data structure, its variants, and applications.

Tools Required

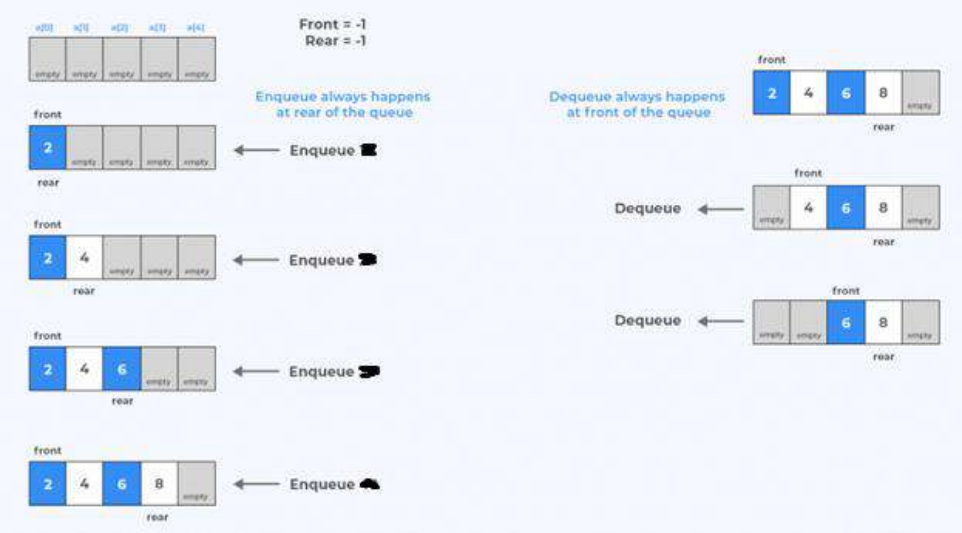
Dev C++ IDE

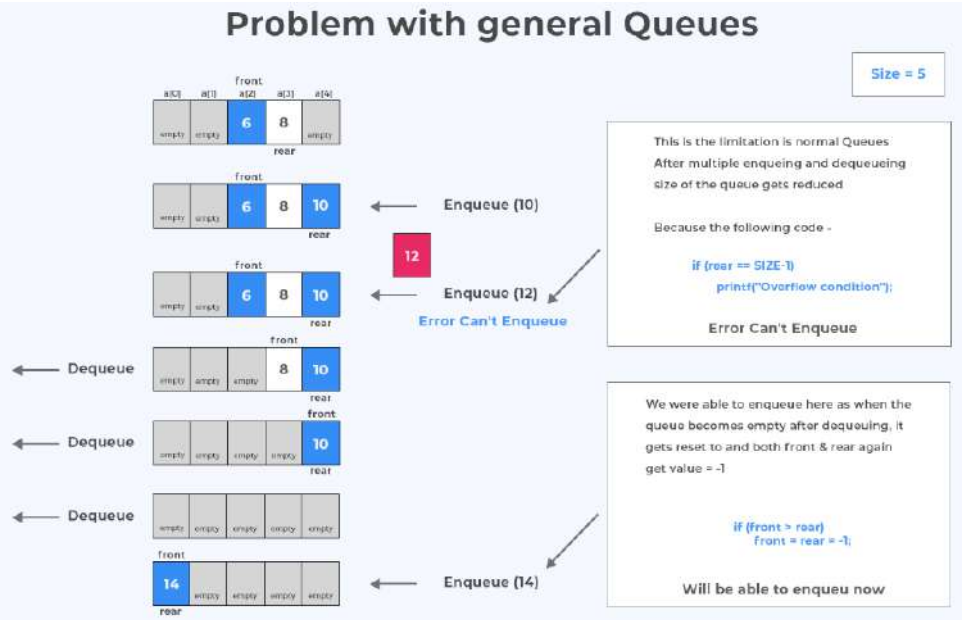
Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

Introduction

A queue is also a linear data structure which works according to FIFO (First IN First Out) manner. Of the two ends of the queue, one is designated as the front – where elements are extracted (operation called dequeue), and another is the rear, where elements are inserted (operation called enqueue). The following operations are needed to properly manage a queue:

- clear()—Clear the queue.
- isEmpty()—Check to see if the queue is empty.
- enqueue(el)—Put the element el at the end of the queue.
- dequeue()—Take the first element from the queue.
- firstEl()—Return the first element in the queue without removing it
- lastEl()—Return the last element in the queue without removing it





Advantages of Linear Queue:

- **Simple Implementation**: Linear queues are easy to implement and understand, making them suitable for simple use cases where the data structure does not require dynamic resizing.

Limitations of Linear Queue:

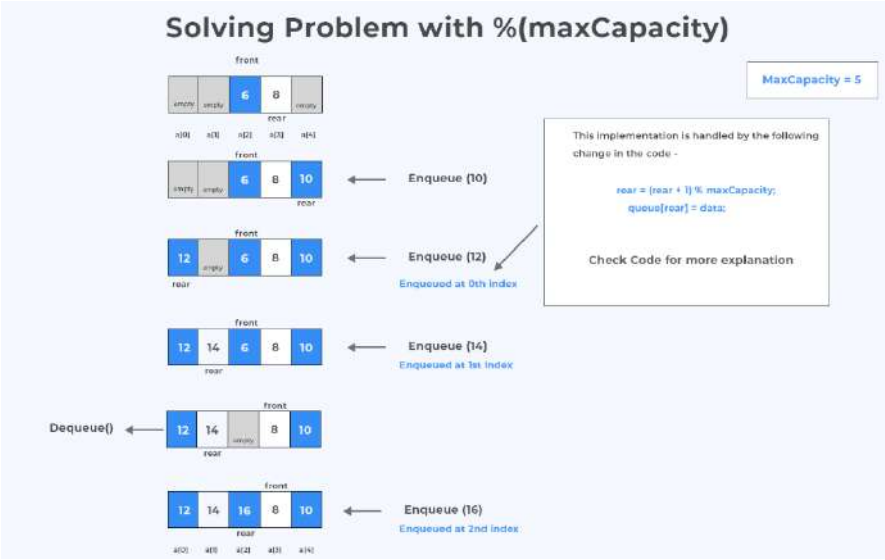
- **Wasted Space**: When elements are dequeued from the front, the space they occupied becomes unusable unless the remaining elements are shifted. This shifting process can be inefficient.
- **Fixed Size**: The size of the array used to implement the queue is fixed, meaning once the maximum capacity is reached, no more elements can be enqueued without resizing the array or creating a new queue.

Example 01: Implementation of Generic Linear Queue Data Structure using Array

```
#include <iostream>
#include <stdexcept>
using namespace std;

template <typename T>
class Queue {
private:
   T* data;                    // Array to store queue elements
   size_t capacity;            // Maximum number of elements the queue can hold
   size_t front;               // Index of the front element
   size_t rear;                // Index of the next position to insert an element

public:
   Queue(size_t initialCapacity = 4) : capacity(initialCapacity), front(-1), rear(-1) {
      data = new T[capacity];                    // Dynamically allocate array for queue elements
   }

    ~Queue() {                                   // Destructor to clean up allocated memory
      delete[] data;     }

   void enqueue(const T& value) {                // Add an element to the rear of the queue
      if (rear == capacity-1) {
         throw overflow_error("Queue is full");   }
      data[++rear] = value;                      // Insert the element at the rear
   }

   void dequeue() {                              // Remove the front element from the queue
      if (isEmpty()) {
         throw out_of_range("Queue is empty");
       }
      front++;                                   // Increment front index
      if(front>rear){
        front=rear=-1;
      }
   }

   T& peek() const {                             // Get the front element of the queue without removing it
      if (isEmpty()) {
         throw out_of_range("Queue is empty");     }
      return data[front+1];                      // Return the front element
   }

   bool isEmpty() const {                        // Check if the queue is empty
      return rear == -1;
   }

   size_t getSize() const {                      // Get the current size of the queue
      return rear;
   }
};

int main() {
   Queue<int> q;
   q.enqueue(1);
   q.enqueue(2);
   q.enqueue(3);
   q.enqueue(4);
   cout << "Front element: " << q.peek() << std::endl;
   q.dequeue();

   cout << "Front element after dequeue: " << q.peek() << std::endl;
   cout << "Queue current size: " << q.getSize() << std::endl;
   return 0;
}
```

Circular Queue

A **circular queue** is a linear data structure that uses a fixed-size array in a circular fashion. In a circular queue, the last position is connected back to the first position to make a circle. This setup allows efficient use of the available space, enabling insertion and deletion operations to be performed in constant time, O(1), if the queue is not full or empty.



Advantages of Circular Queue

- **No Wasted Space**: Unlike linear queues, circular queues make full use of the array's space by treating it as a circular structure. This avoids the problem of wasted space due to shifting elements after a dequeue operation, as seen in linear queues.
- **O(1) Time Complexity**: Both enqueue and dequeue operations can be performed in constant time, O(1), regardless of the number of elements in the queue. This makes circular queues highly efficient for applications that require frequent insertion and removal of elements.

Disadvantages of Circular Queue

- **Fixed Capacity**: If circular queues are implemented using a fixed-size array, the maximum number of elements they can hold is predefined. This means the queue can become full, at which point no more elements can be added without resizing or replacing the queue.
- **Additional Logic for Wrap-Around**: The circular nature requires additional logic to handle the wrap-around of the front and rear pointers. This can make the implementation slightly more complex than a linear queue, especially in managing conditions like queue full or empty.

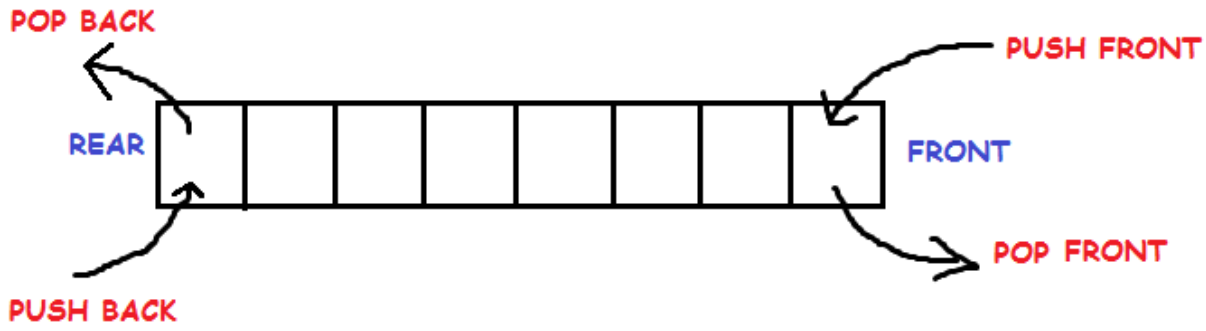
Example 02: Implementation of Generic Circular Queue Data Structure using Array

```
#include <iostream>
#include <stdexcept>
using namespace std;

template <typename T>
class Queue {
private:
   T* data;
   size_t capacity;
   size_t front;
   size_t rear;
   size_t size;

   void resize() {
      size_t newCapacity = capacity * 2;
      T* newData = new T[newCapacity];

      for (size_t i = 0; i < size; ++i) {
         newData[i] = data[(front + i) % capacity];
      }

      delete[] data;
      data = newData;
      capacity = newCapacity;
      front = 0;
      rear = size;
   }

public:
   Queue(size_t initialCapacity = 4) : capacity(initialCapacity), front(0), rear(0), size(0) {
```

```
    data = new T[capacity];
  }

  ~Queue() {
    delete[] data;
  }

  void enqueue(const T& value) {
    if (size == capacity) {
      resize();
    }
    data[rear] = value;
    rear = (rear + 1) % capacity;
    ++size;
  }

  void dequeue() {
    if (isEmpty()) {
      throw out_of_range("Queue is empty");
    }
    front = (front + 1) % capacity;
    --size;
  }

  T& peek() const {
    if (isEmpty()) {
      throw out_of_range("Queue is empty");
    }
    return data[front];
  }

  bool isEmpty() const {
    return size == 0;
  }

  size_t getSize() const {
    return size;
  }
};
int main() {
  Queue<int> q;
  q.enqueue(1);
  q.enqueue(2);
  q.enqueue(3);

  cout << "Front element: " << q.peek() << endl;
  q.dequeue();
  cout << "Front element after dequeue: " << q.peek() << endl;
  cout << "Queue size: " << q.getSize() << endl;
  return 0;
}
```

Double Ended Queue

A double-ended queue (commonly referred to as a deque, pronounced "deck") is a linear data structure that allows insertion and deletion of elements from both ends — the front and the rear. This makes the deque a versatile data structure that can act both as a queue (FIFO) and a stack (LIFO). Managing indices for both ends can make the implementation more complex compared to simpler data structures like stacks or linear queues. Care must be taken to handle edge cases such as deque underflow or overflow.



Applications

1. Print Queue in Operating Systems

In an office setting with multiple users sharing several printers, efficient management of print jobs is crucial. To avoid conflicts and ensure that print jobs are handled in the order they are received, a print queue is implemented.

How It Works:

- **Print Spooler**: A print spooler uses a queue to manage print jobs sent to the printer. When a user submits a print job, the job is placed at the rear of the queue.
- **First In, First Out (FIFO)**: The printer processes jobs in the order they are added to the queue. This FIFO behavior ensures fairness so that jobs are printed in the sequence they are received.

Benefits:

- **Resource Optimization**: Prevents printers from being overloaded with multiple simultaneous requests.
- **User Satisfaction**: Ensures a predictable and fair order of print jobs, reducing user frustration and potential conflicts over printer use.

2. Task Scheduling in Operating Systems

In a multi-user operating system, multiple processes need to share the CPU efficiently. The operating system must ensure that each process gets a fair amount of CPU time to function correctly.

How It Works:

- **Round Robin Scheduling**: One common CPU scheduling algorithm is round robin, where each process is assigned a fixed time slice. The processes are stored in a queue.
- **Process Management**: When a process's time slice expires, it is removed from the front of the queue and placed at the rear, while the next process is moved to the front.
- **Continuous Execution**: This cycle continues, ensuring that all processes get regular CPU time and preventing any single process from monopolizing the CPU.

Benefits:

- **Fairness**: Ensures that all processes receive equal CPU time, preventing any one process from starving others.
- **Responsiveness**: Keeps the system responsive by regularly switching between processes, which is especially important in interactive systems.

3. Customer Service Systems

A call center needs to manage incoming customer calls efficiently to reduce wait times and improve customer satisfaction.

How It Works:

- **Call Queue**: Incoming calls are placed in a queue. Each new call is added to the rear of the queue, and calls are answered in the order they are received.
- **Load Balancing**: The system may use multiple queues to balance loads among available agents.

Calls are dequeued and assigned to agents as they become available.

- **Priority Handling**: Some systems use priority queues, where certain calls (e.g., VIP customers) may be moved ahead in the queue based on predefined rules.

Benefits:

- **Efficient Call Handling**: Ensures that calls are handled in an orderly fashion, reducing the chances of call drops due to long wait times.
- **Improved Customer Satisfaction**: By managing calls effectively, customer wait times are minimized, leading to a better overall experience.

4. Web Server Request Management

Web servers need to handle multiple incoming requests from users browsing the internet. Efficient management of these requests ensures a smooth browsing experience.

How It Works:

- **Request Queue**: When a user sends an HTTP request to a web server, the request is placed in a queue.
- **Thread Pool**: A pool of worker threads dequeues requests from the queue and processes them.
- **Load Management**: If the server is too busy, incoming requests might be queued until a thread becomes available to handle them.

Benefits:

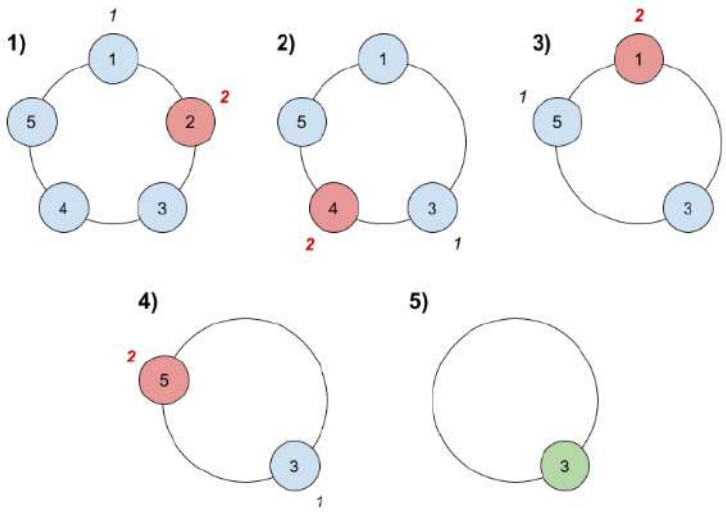
- **Scalability**: Helps in managing high volumes of requests efficiently without overloading the server.
- **Resource Optimization**: Ensures that server resources are used optimally, balancing between serving current requests and managing new ones.

Exercise

1. Design your implementation of the circular double-ended queue (deque). Implement the MyCircularDeque class:
   - MyCircularDeque(int k) Initializes the deque with a maximum size of k.
   - boolean insertFront() Adds an item at the front of Deque. Returns true if the operation is successful, or false otherwise.
   - boolean insertLast() Adds an item at the rear of Deque. Returns true if the operation is successful, or false otherwise.
   - boolean deleteFront() Deletes an item from the front of Deque. Returns true if the operation is successful, or false otherwise.
   - boolean deleteLast() Deletes an item from the rear of Deque. Returns true if the operation is successful, or false otherwise.
   - int getFront() Returns the front item from the Deque. Returns -1 if the deque is empty.
   - int getRear() Returns the last item from Deque. Returns -1 if the deque is empty.
   - boolean isEmpty() Returns true if the deque is empty, or false otherwise.
   - boolean isFull() Returns true if the deque is full, or false otherwise.
   - Main Program:

```
Int main( ){
MyCircularDeque myCircularDeque = new MyCircularDeque(3);
myCircularDeque.insertLast(1);  // return True
myCircularDeque.insertLast(2);  // return True
myCircularDeque.insertFront(3); // return True
myCircularDeque.insertFront(4); // return False, the queue is full.
myCircularDeque.getRear();     // return 2
myCircularDeque.isFull();      // return True
myCircularDeque.deleteLast();   // return True
myCircularDeque.insertFront(4); // return True
myCircularDeque.getFront();     // return 4 }
```

2. There are n friends that are playing a game. The friends are sitting in a circle and are numbered from 1 to n in clockwise order. More formally, moving clockwise from the ith friend brings you to the (i+1)th friend for 1 <= i < n, and moving clockwise from the nth friend brings you to the 1st friend. The rules of the game are as follows:
   - Start at the 1st friend.
   - Count the next k friends in the clockwise direction including the friend you started at. The counting wraps around the circle and may count some friends more than once.
   - The last friend you counted leaves the circle and loses the game.
   - If there is still more than one friend in the circle, go back to step 2 starting from the friend immediately clockwise of the friend who just lost and repeat.
   - Else, the last friend in the circle wins the game.
   - Given the number of friends, n, and an integer k, return the winner of the game.




```
Input: n = 5, k = 2
Output: 3
```

3. You are designing a simple customer service call center simulation. The call center has a fixed number of customer service representatives (CSRs), and it needs to manage incoming calls efficiently. You will implement a call queue system where calls are handled on a first-come, first-served basis.

Requirements:

- Queue for Incoming Calls: Use a queue to manage incoming customer calls.
- Fixed Number of CSRs: The call center has a fixed number of CSRs. Each CSR can handle one call at a time.

Instructions:

- Define the class for a call with attributes like Call ID, Arrival Time, and Customer Name, and a constructor to initialize these attributes.
- Define another class for simulating a call center
- Define a queue as a data member to hold incoming calls, an int CSR for defining the number of CSRs and a bool vector for defining whether a CSR is available or not
- Create a function to add a new call to the queue.
- Create a function to process calls, assigning them to available CSRs.
- Simulate the processing time of each call and update the status of the call once completed.
- Initialize the call center with a fixed number of CSRs.
- Simulate the arrival of calls at different times.
- Process calls based on CSR availability and handle calls in a first-come, first-served manner.

4. Design an algorithm that accepts a stream of integers and retrieves the product of the last k integers of the stream. Implement the ProductOfNumbers class:
   - ProductOfNumbers() Initializes the object with an empty stream.
   - void add(int num) Appends the integer num to the stream.
   - int getProduct(int k) Returns the product of the last k numbers in the current list. You can assume that always the current list has at least k numbers.
   - The test cases are generated so that, at any time, the product of any contiguous sequence of numbers will fit into a single 32-bit integer without overflowing.
   - Main Program

```
Int main(){
ProductOfNumbers productOfNumbers = new ProductOfNumbers();
productOfNumbers.add(3);      // [3]
productOfNumbers.add(0);      // [3,0]
productOfNumbers.add(2);      // [3,0,2]
productOfNumbers.add(5);      // [3,0,2,5]
productOfNumbers.add(4);      // [3,0,2,5,4]
```

```
productOfNumbers.getProduct(2); // return 20. The product of the last 2 numbers is 5 * 4 = 20
productOfNumbers.getProduct(3); // return 40. The product of the last 3 numbers is 2 * 5 * 4 = 40
productOfNumbers.getProduct(4); // return 0. The product of the last 4 numbers is 0 * 2 * 5 * 4 = 0
productOfNumbers.add(8);       // [3,0,2,5,4,8]
productOfNumbers.getProduct(2); // return 32. The product of the last 2 numbers is 4 * 8 = 32}
```

5. For a stream of integers, implement a data structure that checks if the last k integers parsed in the stream are equal to value. Implement the DataStream class:
   - DataStream(int value, int k) Initializes the object with an empty integer stream and the two integers value and k.
   - boolean consec(int num) Adds num to the stream of integers. Returns true if the last k integers are equal to value, and false otherwise. If there are less than k integers, the condition does not hold true, so returns false.
   - Main Program:

```
void main(){
DataStream dataStream = new DataStream(4, 3); //value = 4, k = 3
dataStream.consec(4); // Only 1 integer is parsed, so returns False.
dataStream.consec(4); // Only 2 integers are parsed.
                      // Since 2 is less than k, returns False.
dataStream.consec(4); // The 3 integers parsed are all equal to value, so returns True.
dataStream.consec(3); // The last k integers parsed in the stream are [4,4,3].
                      // Since 3 is not equal to value, it returns False.
}
```

Lab 03 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		

Data Structure Algorithms & Applications (CT-159)
Lab 04

Elementary Sorting Techniques

Objectives

The objective of the lab is to get students familiar with Bubble sort, insertion sort, and selection sort techniques.

Tools Required

Dev C++ IDE

Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

Introduction

A sorting algorithm is a procedure used to arrange the elements of a list in a particular order, typically either ascending or descending. Sorting algorithms optimize other algorithms that require sorted data, like search algorithms or data analysis techniques.

Selection Sort

Selection sort is a simple, comparison-based sorting algorithm that divides the input array into two parts: the sorted part and the unsorted part. The sorted part is built up from left to right at the front of the array, while the unsorted part occupies the rest. The algorithm repeatedly selects the

smallest (or largest, depending on sorting order) element from the unsorted part and swaps it with the first element of the unsorted part, thus expanding the sorted part by one element.

Selection Sort Algorithm

1. Start with the first element (index 0) and consider it the minimum.
2. Search the unsorted part of the array to find the smallest element.
3. Swap the smallest element with the first element of the unsorted part.
4. Move the boundary between the sorted and unsorted parts one element to the right.
5. Repeat the process for the next position until the entire array is sorted.

Time Complexity: Best case: $O(n^2)$, Worst case: $O(n^2)$, Average case: $O(n^2)$

Space Complexity: O(1) (In-place sorting)

Properties:

- Not stable: It does not preserve the relative order of elements with equal keys.
- Not adaptive: It performs the same number of comparisons regardless of the input's initial order.

Selection sort is easy to implement but inefficient on large lists, making it more suitable for small datasets or educational purposes.

Example 01: A C++ program for implementation of selection sort

```
#include <iostream>
using namespace std;
//Swap function
void swap(int &xp, int &yp){
	int temp = xp;
	xp = yp;
	yp = temp;
}
void selectionSort(int* arr, int n){
	int i, j, min_idx;
	// One by one move boundary of unsorted subarray
	for (i = 0; i < n-1; i++){
	// Find the minimum element in unsorted array
		min_idx = i;
		for (j = i+1; j < n; j++)
			if (arr[j] < arr[min_idx])
				min_idx = j;
		// Swap the found minimum element with the first element
		if(min_idx!=i)
			swap(arr[min_idx], arr[i]);  }
}
//Function to print an array
void printArray(int *arr, int size){
	int i;
	for (i=0; i < size; i++)
		cout << arr[i] << " ";
	cout << endl;  }
// Driver program to test above functions
int main(){
	int arr[] = {64, 25, 12, 22, 55, 4, 11};
	int n = sizeof(arr)/sizeof(arr[0]);
	selectionSort(arr, n);
	cout << "Sorted array: \n";
```

```
        printArray(arr, n);
        return 0;
}
```

Bubble Sort

Bubble sort is a simple, comparison-based sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process is repeated until the list is sorted. The algorithm gets its name because larger elements "bubble" to the top (end of the list) with each pass through the array.

Bubble Sort Algorithm

1. Start at the beginning of the array.
2. Compare each pair of adjacent elements.
3. If the current element is greater than the next one (for ascending order), swap them.
4. Repeat the process for each pair until you reach the end of the list, ensuring the largest element is in its correct position.
5. After each complete pass through the list, the next largest element will have moved to its correct position, so the next pass can ignore the last sorted elements.
6. Continue repeating passes until no swaps are needed, indicating the list is fully sorted.

Time Complexity: Best case (already sorted): O(n) (optimized version that stops early if no swaps are made), Worst case: $O(n^2)$, Average case: $O(n^2)$

Space Complexity: O(1) (In-place sorting)

Properties:

- Stable: It preserves the relative order of elements with equal keys.
- Not adaptive (in its basic form): Although an optimized version can detect an already sorted array, the standard algorithm does not take advantage of existing order.

Bubble sort is primarily useful for educational purposes and small datasets, but it's inefficient for large datasets due to its quadratic time complexity.

Example 02: A C++ program for implementation of Bubble sort

```
#include <iostream>
using namespace std;
// A function to implement bubble sort
void bubbleSort(int* arr, int n){
        int i, j;
        for (i = 0; i < n - 1; i++)
                // Last i elements are already in place
                for (j = 0; j < n - i - 1; j++)
                        if (arr[j] > arr[j + 1])
                                swap(arr[j], arr[j + 1]);
}
// Function to print an array
void printArray(int* arr, int size){
        int i;
        for (i = 0; i < size; i++)
                cout << arr[i] << " ";
        cout << endl;  }
// Driver code
int main(){
        int arr[] = { 5, 1, 4, 2, 8};
        int N = sizeof(arr) / sizeof(arr[0]);
        bubbleSort(arr, N);
        cout << "Sorted array: \n";
        printArray(arr, N);
        return 0;
}
```

Insertion sort

Insertion sort is a straightforward, comparison-based sorting algorithm that builds the sorted array one element at a time. It works by repeatedly taking one element from the unsorted portion of the array and inserting it into its correct position within the sorted portion.

Insertion Sort Algorithm

1. Start with the second element (index 1) of the array. Consider the first element (index 0) as the sorted portion.
2. Compare the current element with the elements in the sorted portion.
3. Shift all elements in the sorted portion that are larger than the current element to the right.
4. Insert the current element into its correct position.
5. Repeat this process for each element in the unsorted portion until the entire array is sorted.

Time Complexity: Best case: O(n) (when the array is already sorted), Worst case: $O(n^2)$ (when the array is sorted in reverse order), Average case: $O(n^2)$

Space Complexity: O(1) (In-place sorting)

Properties:

- Stable: It preserves the relative order of elements with equal keys.
- Adaptive: Performs better on nearly sorted or small data sets.

Insertion sort is most efficient for small or nearly sorted data, and it is commonly used as a building block for more complex algorithms like Timsort.

Example 03: A C++ program for insertion sort

```
#include <iostream>
using namespace std;
// Function to sort an array using insertion sort
void insertionSort(int* arr, int n){
	int i, key, j;
	for (i = 1; i < n; i++){
		key = arr[i];
		j = i - 1;
	// Move elements of arr[0..i-1], that are greater than
	// key, to one position ahead of their current position
		while (j >= 0 && arr[j] > key){
			arr[j + 1] = arr[j];
			j = j - 1;
		}
		arr[j + 1] = key;
	}
}
void printArray(int* arr, int n){
	int i;
	for (i = 0; i < n; i++)
		cout << arr[i] << " ";
	cout << endl;
}
int main(){
	int arr[] = { 12, 11, 13, 5, 6 };
	int N = sizeof(arr) / sizeof(arr[0]);
	insertionSort(arr, N);
	printArray(arr, N);
	return 0;
}
```

Visualization of the above mentioned sorting techniques is available at :
https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html

Exercise

1. If the array is already sorted, we don't want to continue with the comparisons. This can be achieved with modified bubble sort. Update the code in example 02 to have a modified bubble sort function.

2. Given an array **arr[ ]** of length **N** consisting cost of **N** toys and an integer **K** the amount with you. The task is to find maximum number of toys you can buy with **K** amount.
   Test Case: Input: N = 7, K = 50, arr[] = {1, 12, 5, 111, 200, 1000, 10}, **Output:** 4
   Explanation: The costs of the toys. You can buy are 1, 12, 5 and 10.

3. Create a single class Sort, which will provide the user the option to choose between all 3 sorting techniques. The class should have following capabilities:
- Take an array and a string (indicating the user choice for sorting technique) as input and perform the desired sorting.
- Should allow the user to perform analysis on a randomly generated array. The analysis provides number of comparisons and number of swaps performed for each technique.
- After printing all the results in the main program, highlight the best and worst techniques.

4. Given an array of integers arr, sort the array by performing a series of **pancake flips**. In one pancake flip we do the following steps:
- Choose an integer k where 1 <= k <= arr.length.
- Reverse the sub-array arr[0...k-1] (**0-indexed**).
  For example, if arr = [3,2,1,4] and we performed a pancake flip choosing k = 3, we reverse the sub-array [3,2,1], so arr = [1,2,3,4] after the pancake flip at k = 3. Return *an array of the* k-*values corresponding to a sequence of pancake flips that sort* arr. Any valid answer that sorts the array within 10 * arr.length flips will be judged as correct.
  Example 1: Input: arr = [3,2,4,1], **Output:** [4,2,4,3]
  Explanation: We perform 4 pancake flips, with k values 4, 2, 4, and 3.
  Starting state: arr = [3, 2, 4, 1]
  After 1st flip (k = 4): arr = [1, 4, 2, 3]
  After 2nd flip (k = 2): arr = [4, 1, 2, 3]
  After 3rd flip (k = 4): arr = [3, 2, 1, 4]
  After 4th flip (k = 3): arr = [1, 2, 3, 4], which is sorted.

5. Given an array nums with n objects colored red, white, or blue, sort them inplace so that objects of the same color are adjacent, with the colors in the order red, white, and blue. We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively. You must solve this problem by writing a sort function.
   Example 1: Input: nums = [2,0,2,1,1,0], **Output:** [0,0,1,1,2,2]
   Example 2: Input: nums = [2,0,1], **Output:** [0,1,2]

Lab 04 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		

Data Structure Algorithms & Applications (CT-159)
Lab 05

Advanced Sorting Techniques

Objectives

The objective of the lab is to get students familiar with Advanced sorting techniques which include quick sort and merge sort.

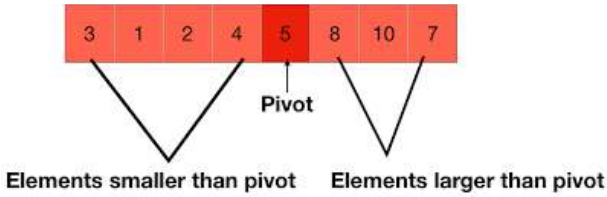
Tools Required

Dev C++ IDE

Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

Quick Sort Algorithm

Quicksort is an application of sorting using stacks. It also uses divide-and-conquer approach. In quicksort, a pivot element is selected and positioned in such a manner that all the elements smaller than the pivot element are to its left and all the elements greater than the pivot are to its right. The elements to the left and right of the pivot form two separate arrays and are sorted using this same approach. This continues until each subarray consists of a single element. The elements are merged to get the required sorted list. The pivot element can be the first, last, random, or median value in the given list.



Time Complexity: O(n log n) on average, O(n^2) in the worst case.
Space Complexity: O(log n) for recursive calls.
Stability: Not stable by default.
Aglorithm main steps:

1. Pick a pivot element.
2. Partition the array such that elements less than the pivot go to the left and greater elements go to the right.
3. Recursively apply the process to the subarrays.

Applications: Often used in practice due to its in-place sorting and good average performance.

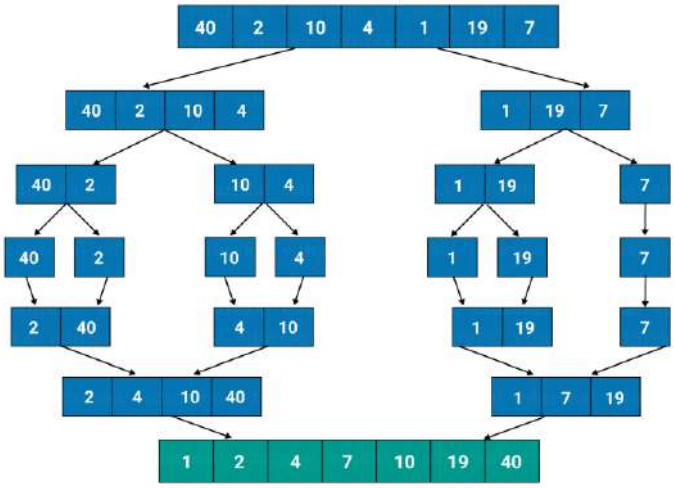
Example 01: Recursive Implementation of Quick Sort Algorithm

```
#include <iostream>
using namespace std;

// Function to swap two elements
void swap(int& a, int& b) {
   int temp = a;
   a = b;
   b = temp;
}

// Partition function: places the pivot element in its correct position
int partition(int arr[], int low, int high) {
   int pivot = arr[high]; // Choosing the last element as pivot
   int i = low - 1; // Index of smaller element

   for (int j = low; j < high; j++) {
      // If the current element is smaller than or equal to the pivot
      if (arr[j] <= pivot) {
         i++; // Move the smaller element's index forward
         swap(arr[i], arr[j]);
      }
   }
   // Place the pivot in its correct position
   swap(arr[i + 1], arr[high]);
   return i + 1; // Return the partition index
}

// Quick Sort function
void quickSort(int arr[], int low, int high) {
   if (low < high) {
      // Partition the array
      int pi = partition(arr, low, high);

      // Recursively sort elements before and after partition
      quickSort(arr, low, pi - 1);
      quickSort(arr, pi + 1, high);
   }
}

// Function to print the array
void printArray(int arr[], int size) {
   for (int i = 0; i < size; i++) {
      cout << arr[i] << " ";
```

```
    }
    cout << endl;
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Unsorted array: ";
    printArray(arr, n);

    // Perform Quick Sort
    quickSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    printArray(arr, n);

    return 0;
}
```

Merge Sort Algorithm

Merge Sort is a divide-and-conquer algorithm. It divides the input array into smaller subarrays, recursively sorts them, and then merges the sorted subarrays back together.



Time Complexity: O(n log n)
Space Complexity: O(n)
Stability: Stable (preserves the relative order of equal elements).
Algorithm main Steps:

1. Split the array into two halves.
2. Recursively sort each half.
3. Merge the two sorted halves into one sorted array.

Applications: Works well with linked lists and large datasets where stability is required.

Example 02: Recursive Implementation of Merge Sort Algorithm

```
#include <iostream>
using namespace std;

// Function to merge two subarrays
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1; // Size of the left subarray
    int n2 = right - mid;    // Size of the right subarray

    // Create temporary arrays
    int L[n1], R[n2];

    // Copy data to the temporary arrays L[] and R[]
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    // Merge the temporary arrays back into arr[left..right]
```

```
    int i = 0; // Initial index of the left subarray
    int j = 0; // Initial index of the right subarray
    int k = left; // Initial index of the merged subarray

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[], if any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[], if any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

// Function to implement Merge Sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        // Find the middle point
        int mid = left + (right - left) / 2;

        // Recursively sort first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

// Main function
int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int arr_size = sizeof(arr) / sizeof(arr[0]);

    cout << "Unsorted array: ";
    printArray(arr, arr_size);

    // Call mergeSort
    mergeSort(arr, 0, arr_size - 1);

    cout << "Sorted array: ";
    printArray(arr, arr_size);

    return 0;
}
```

Exercise

1. Implement non-recursive version of quick sort algorithm using stack.
2. Implement non-recursive version of the merge sort algorithm.
3. Given an array of intervals where intervals[i] = [starti, endi], merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.
   Example 1: Input: intervals = [[1,3],[2,6],[8,10],[15,18]], Output: [[1,6],[8,10],[15,18]]
   Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].
   Example 2: Input: intervals = [[1,4],[4,5]], Output: [[1,5]]
   Explanation: Intervals [1,4] and [4,5] are considered overlapping.
4. We define a harmonious array as an array where the difference between its maximum value and its minimum value is exactly 1. Given an integer array nums, return the length of its longest harmonious subsequence among all its possible subsequences. A subsequence of array is a sequence that can be derived from the array by deleting some or no elements without changing the order of the remaining elements.
   Example 1: Input: nums = [1,3,2,2,5,2,3,7], Output: 5
   Explanation: The longest harmonious subsequence is [3,2,2,2,3].
   Example 2: Input: nums = [1,2,3,4], Output: 2
   Example 3: Input: nums = [1,1,1,1], Output: 0
5. There is a car with capacity empty seats. The vehicle only drives east (i.e., it cannot turn around and drive west). You are given the integer capacity and an array trips where trips[i] = [numPassengersi, fromi, toi] indicates that the ith trip has numPassengersi passengers and the locations to pick them up and drop them off are fromi and toi respectively. The locations are given as the number of kilometers due east from the car's initial location. Return true if it is possible to pick up and drop off all passengers for all the given trips, or false otherwise.
   Example 1: Input: trips = [[2,1,5],[3,3,7]], capacity = 4, Output: false
   Example 2: Input: trips = [[2,1,5],[3,3,7]], capacity = 5, Output: true

Lab 05 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		

Data Structure Algorithms & Applications (CT-159)
Lab 06

Linked List

Objectives

The objective of the lab is to get students familiar with Linked List and its operation, that is, insertion and deletion.

Tools Required

Dev C++ IDE

Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

Introduction

A **linked list** is a linear data structure in which elements (also called nodes) are stored in sequence but are not stored in contiguous memory locations. Instead, each element points to the next one in the sequence. This allows dynamic memory allocation, meaning that linked lists can easily grow or shrink during runtime.

Types of Linked Lists:

- **Singly Linked List**: Each node contains data and a pointer/reference to the next node.
- **Doubly Linked List**: Each node contains data, a pointer to the next node, and a pointer to the previous node.
- **Circular Linked List**: Similar to a singly or doubly linked list, but the last node points back to the first node, forming a loop.

Properties of Linked Lists:

- **Dynamic Size**: Linked lists do not have a fixed size, and their length can change dynamically as nodes are inserted or deleted.
- **Efficient Insertions and Deletions**: Adding or removing nodes from a linked list is efficient (especially at the head and tail) since no elements need to be shifted.
- **Non-contiguous Memory Allocation**: Nodes can be scattered in memory, unlike arrays, where elements are stored in contiguous blocks.
- **Pointers/References**: Each node contains at least one pointer that refers to another node, creating the link between nodes.
- **No Random Access**: Unlike arrays, linked lists do not allow direct access to an element by index. Traversal from the head is required to access specific nodes.

```
#include <iostream>
using namespace std;
class Node {   // Node class representing each element in the linked list
public:
   int data;  // Data of the node
   Node* next; // Pointer to the next node

   Node(int data) {    // Constructor to initialize the node with data and set next to nullptr
      this->data = data;
      this->next = nullptr;
   }
};
class LinkedList {              // LinkedList class representing the singly linked list
private:
   Node* head; // Pointer to the first node in the list
public:
   LinkedList() {                  // Constructor to initialize the linked list
      head = nullptr;
   }

   void insert(int data) {      // Insert a new node at the end of the list
      Node* newNode = new Node(data); // Create a new node
      if (head == nullptr) {
         head = newNode;  // If list is empty, set head to the new node
      } else {
         Node* temp = head;
         while (temp->next != nullptr) {
            temp = temp->next;  // Traverse to the last node
         }
         temp->next = newNode;  // Set the next of the last node to new node
      }
   }
```

```
    void deleteNode(int key) {      // Delete the first node with the given value
        Node* temp = head;
        Node* prev = nullptr;

        if (temp != nullptr && temp->data == key) {                // If head node itself holds the key
            head = temp->next; // Move head to the next node
            delete temp;      // Free the old head
            return;
        }

        while (temp != nullptr && temp->data != key) {           // Search for the key to be deleted
            prev = temp;
            temp = temp->next;
        }
        if (temp == nullptr) return;    // If key was not present in the list
        prev->next = temp->next;     // Unlink the node from the linked list
        delete temp; // Free the memory of the node
    }

    bool search(int key) {   // Search for a node with a given value
        Node* temp = head;
        while (temp != nullptr) {
            if (temp->data == key) {
                return true;  // Key found
            }
            temp = temp->next;
        }
        return false;  // Key not found
    }

    void display() {               // Display the contents of the linked list
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};
int main() {
    LinkedList list;

    list.insert(10); // Insert elements into the linked list
    list.insert(20);
    list.insert(30);
    list.insert(40);

    cout << "Linked List: ";
    list.display();

    int key = 20;   // Search for an element
    if (list.search(key)) {
        cout << key << " found in the list." << endl;
    } else {
        cout << key << " not found in the list." << endl;
    }

    list.deleteNode(20); // Delete an element
    cout << "After deleting 20: ";
    list.display();
    return 0;
}
```

Computational Complexity:

- **Access Time**: O(n), since you need to traverse from the head to the desired node.
- **Search**: O(n), since traversal is required to search for a specific element.
- **Insertion**:
  At the Head: O(1), constant time since you just need to update the pointers.
  At the Tail: O(n), as traversal is required to reach the end.
  In the Middle: O(n), since traversal to the insertion point is required.
- **Deletion**:
  At the Head: O(1), constant time as it involves updating pointers.
  At the Tail: O(n), since traversal is needed to find the previous node.
  In the Middle: O(n), because you must first find the node to delete by traversal.

Summary of Complexity Comparison:

Operation	Array (Static/Dynamic)	Singly Linked List
Insert at Start	$O(n)$	$O(1)$
Insert at End	Amortized $O(1)$ / $O(n)$	$O(n)$
Insert in Middle	$O(n)$	$O(n)$
Delete at Start	$O(n)$	$O(1)$
Delete at End	$O(1)$	$O(n)$
Delete in Middle	$O(n)$	$O(n)$

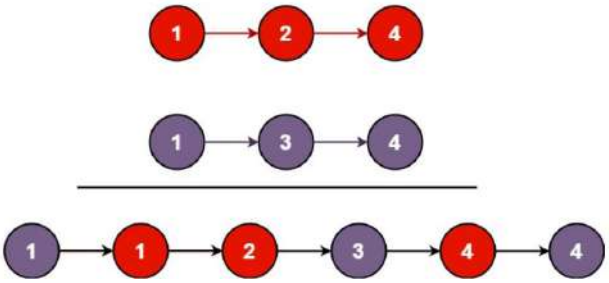
Key Use Cases:

- When dynamic memory allocation is needed.
- When frequent insertions and deletions are required, especially at the head or middle of the list.
- When the size of the dataset is unknown in advance.

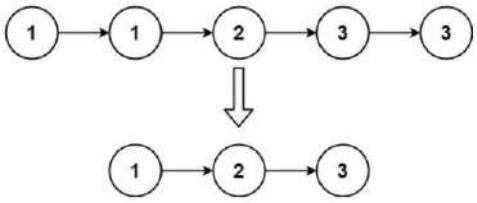
Linked lists are often used in scenarios like implementing stacks, queues, and adjacency lists in graphs.

Exercise

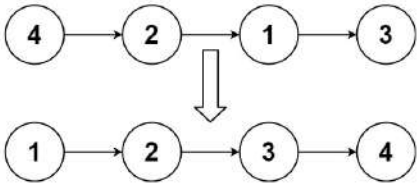
1. You are given the heads of two sorted linked lists list1 and list2. Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists. Return the head of the merged linked list.
   Example: **Input:** list1 = [1,2,4], list2 = [1,3,4], **Output:** [1,1,2,3,4,4]



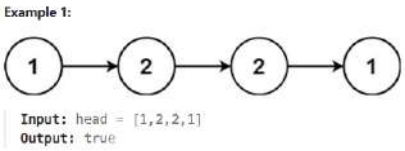
2. Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.



3. Given the head of a linked list, return the list after sorting it in ascending order. Use mergesort.



4. Given the head of a singly linked list, return true if it is a palindrome or false otherwise.



5. Implement class of a Circular Queue using a Linked List.
6. Implement class of a Stack using a Linked List.

Lab 06 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		

Data Structure Algorithms & Applications (CT-159)
Lab 07

Types of Linked List

Objectives

The objective of the lab is to get students familiar with Circular and Doubly Linked List.

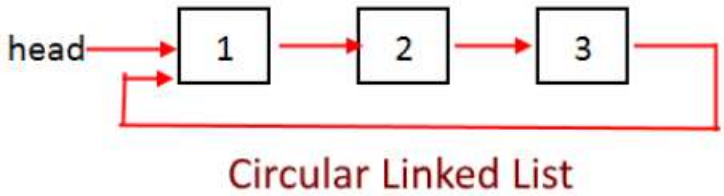
Tools Required

Dev C++ IDE

Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

Circular Linked List

A circular linked list is a variation of a linked list where the last node points back to the first node, creating a circular structure. This allows for continuous traversal of the list without ever encountering a nullptr, as is the case with linear linked lists.



Properties of a Circular Linked List:

- Circular Nature: The next pointer of the last node points to the first node, forming a circular loop.
- No nullptr at the End: Unlike a singly linked list, where the last node points to nullptr, a circular linked list loops back to the first node.
- Traversal: You can start at any node and traverse the entire list. It's common to use an additional condition to stop traversing (such as visiting the starting node again).
- Single Entry Point: Usually, a circular linked list is referenced by a pointer to the last node (so that both the head and tail can be accessed efficiently) or the head node.
- Efficient Operations: Insertion and deletion operations are efficient, particularly if the list is used in scenarios like a queue or round-robin scheduling.

```
#include <iostream>
using namespace std;

// Node structure
class Node {
        public:
  int data;
  Node* next;
  Node(int d){
        data=d;
        next=NULL;
        }
};

// Class to represent a circular linked list
class CircularLinkedList {
private:
  Node* tail; // Points to the tail node in the list

public:
  CircularLinkedList() : tail(NULL) {}

  void insertAtEnd(int value) {
    Node* newNode = new Node(value);
    if (tail == NULL) {     // List is empty, create a single node circular list
      tail = newNode;
      tail->next = tail;
    } else {
      // Insert after the tail node
      newNode->next = tail->next;  // Point new node to head
      tail->next = newNode;       // Update tail node's next
      tail = newNode;            // Update the tail pointer
    }
  }

  void insertAtBeginning(int value) {
    Node* newNode = new Node(value);
    if (tail == NULL) {           // List is empty, create a single node circular list
      tail = newNode;
      tail->next = tail;
```

```
    } else {          // Insert new node after the tail node and update head
        newNode->next = tail->next;
        tail->next = newNode;
    }
}

void display() const {
    if (tail == NULL) {
        cout << "List is empty." << endl;
        return;
    }

    Node* temp = tail->next;  // Start from the head
    do {
        cout << temp->data << " ";
        temp = temp->next;
    } while (temp != tail->next);  // Stop when we reach the head again

    cout << endl;
}

void deleteNode(int key) {
    if (tail == NULL) {
        cout << "List is empty, cannot delete." << endl;
        return;
    }

    // Single node case
    if (tail == tail->next && tail->data == key) {
        delete tail;
        tail = NULL;
        return;
    }

    // Traverse the list to find the node to be deleted
    Node* temp = tail->next;
    Node* prev = tail;

    // Case: Deleting the head node
    if (temp->data == key) {
        prev->next = temp->next;
        delete temp;
        return;
    }

    // General case: find the node to delete
    do {
        prev = temp;
        temp = temp->next;
    } while (temp != tail->next && temp->data != key);

    if (temp->data == key) {
        prev->next = temp->next;
        if (temp == tail) {
            tail = prev;  // If the tail node is deleted, update the tail pointer
        }
        delete temp;
    } else {
        cout << "Node with value " << key << " not found." << endl;
    }
}

~CircularLinkedList() {
    if (tail != NULL) {
        Node* current = tail->next;
        Node* nextNode;
        while (current != tail) {
            nextNode = current->next;
            delete current;
```

```
            current = nextNode;
        }
        delete tail;
    }
  }
};

int main() {
  CircularLinkedList cll;

  cll.insertAtEnd(10);
  cll.insertAtEnd(20);
  cll.insertAtEnd(30);
  cll.insertAtBeginning(5);

  cout << "Circular Linked List: ";
  cll.display();

  cll.deleteNode(20);
  cout << "After deleting 20: ";
  cll.display();

  return 0;
}
```

Circular vs. Singly Linked List

The circular linked list offers several advantages over a singly linked list in certain use cases, mainly because of its circular nature, which provides continuous traversal and efficient operations. Here are some key advantages:

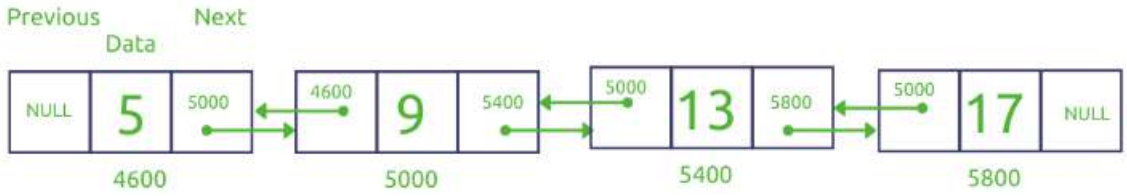
- Efficient Continuous Traversal:
  - In a circular linked list, after reaching the last node, traversal doesn't have to stop at a nullptr; instead, it loops back to the first node. This makes it useful in scenarios where continuous cycling through the list is required, such as: Round-robin scheduling (e.g., time-sharing in operating systems), Buffer management (e.g., circular buffers)
  - In a singly linked list, after reaching the end, traversal stops, requiring an explicit loop restart from the head.
- Easier Insertions at Both Ends:
  - In a circular linked list, maintaining a pointer to the last node allows easy access to both the head (first node) and the tail (last node) of the list. This simplifies insertion at both the beginning and the end of the list without needing to traverse the entire list to reach the last node.
  - In a singly linked list, you would need to traverse the list to insert at the end, making it less efficient, especially for long lists.
- Efficient Queue Implementation:
  - A circular linked list is ideal for implementing queues (especially circular queues) because it naturally supports enqueue and dequeue operations at both ends. By having the last node point to the first, you avoid the need to reset pointers manually or handle edge cases when the list reaches the end.
  - In a singly linked list, implementing a queue may require additional bookkeeping for managing the head and tail.

Example Use Cases: In summary, circular linked lists offer advantages in scenarios requiring continuous or repetitive processing, efficient insertions at both ends, and fixed memory usage without losing data integrity.

- Circular queues in memory-constrained systems (e.g., real-time operating systems).
- Round-robin task scheduling (e.g., operating system schedulers, multiplayer games).
- Circular buffers for data streams or buffering audio/video.

Doubly Linked List

A **doubly linked list** is a type of linked list in which each node contains a data element and two pointers: one pointing to the next node and another pointing to the previous node. This allows traversal in both forward and backward directions, offering more flexibility than a singly linked list.



Properties of a Doubly Linked List:

- **Two Pointers**: Each node has two pointers, one to the next node and one to the previous node, allowing bi-directional traversal.
- **Bi-directional Traversal**: You can traverse the list both forwards and backwards.
- **Efficient Insertions and Deletions**: You can insert or delete nodes more efficiently at any position, especially when you have a reference to the node, because you have access to the previous node without needing to traverse the list.
- **More Memory Overhead**: Since each node stores an extra pointer (prev), the memory overhead is higher than a singly linked list.

Advantages of a Doubly Linked List:

- **Bi-directional Traversal**: You can traverse the list in both directions (from head to tail or tail to head), making it more versatile for certain algorithms and data structures (e.g., implementing a deque).
- **Efficient Deletion and Insertion**: When deleting or inserting a node at an arbitrary position, a doubly linked list allows quick access to both neighboring nodes, avoiding the need to traverse the list to find the previous node.
- **Backtracking**: You can easily backtrack to the previous node, which is useful in certain applications like undo features or iterators that require two-way movement.

Disadvantages:

- **Memory Overhead**: Each node requires extra memory for the prev pointer.
- **Slightly More Complex**: Maintaining the prev pointer in insertion and deletion operations adds complexity compared to a singly linked list.

```
#include <iostream>
using namespace std;

// Node structure for the doubly linked list
class Node {
   public:
            int data;
   Node* next;
   Node* prev;
   Node(int d){
            data=d;
            next=NULL;
            prev=NULL;
            }
};

class DoublyLinkedList {
private:
   Node* head;  // Pointer to the head of the list

public:
   // Constructor
   DoublyLinkedList() : head(NULL) {}

   void insertAtFront(int value) {
      Node* newNode = new Node(value);
      newNode->next = head;
      newNode->prev = NULL;
```

```
    if (head != NULL) {
        head->prev = newNode;  // Update previous pointer of the old head
    }
    head = newNode;
}

void insertAtEnd(int value) {
    Node* newNode = new Node(value);
    newNode->next = NULL;

    if (head == NULL) {            // If the list is empty, make the new node the head
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    // Otherwise, find the last node
    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    // Insert the new node at the end
    temp->next = newNode;
    newNode->prev = temp;
}

void deleteNode(int value) {     // Delete a node from the list by value
    Node* temp = head;

    // Traverse the list to find the node with the given value
    while (temp != NULL && temp->data != value) {
        temp = temp->next;
    }

    if (temp == NULL) {        // Node not found
        cout << "Node with value " << value << " not found." << endl;
        return;
    }

    if (temp->prev != NULL) {                // Update the previous node's next pointer
        temp->prev->next = temp->next;
    } else {
        // Node is the head
        head = temp->next;
    }

    if (temp->next != NULL) {               // Update the next node's previous pointer
        temp->next->prev = temp->prev;
    }

    // Free the memory of the node
    delete temp;
}

void displayForward() const {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

void displayBackward() const {
    Node* temp = head;

    // Traverse to the end of the list
```

```
        if (temp == NULL) return;
        while (temp->next != NULL) {
            temp = temp->next;
        }

        // Now traverse backward from the last node
        while (temp != NULL) {
            cout << temp->data << " ";
            temp = temp->prev;
        }
        cout << endl;
    }

    // Destructor to free the memory
    ~DoublyLinkedList() {
        Node* temp = head;
        while (temp != NULL) {
            Node* next = temp->next;
            delete temp;
            temp = next;
        }
    }
};

int main() {
    DoublyLinkedList dll;

    dll.insertAtFront(10);
    dll.insertAtFront(20);
    dll.insertAtEnd(30);
    dll.insertAtEnd(40);

    cout << "Doubly Linked List (Forward): ";
    dll.displayForward();

    cout << "Doubly Linked List (Backward): ";
    dll.displayBackward();

    dll.deleteNode(20);
    cout << "After deleting 20, list (Forward): ";
    dll.displayForward();

    return 0;
}
```

Exercise

1. Implement class of a Circular Queue using a circular Linked List.
2. Implement class of a double ended queue using doubly Linked List.
3. Create two doubly link lists, say L and M . List L should be containing all even elements from 2 to 10 and list M should contain all odd elements from 1 to 9. Create a new list N by concatenating list L and M.
4. Sort the contents of list N created in Q4 in descending order.
5. You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

- Implement the BrowserHistory class: BrowserHistory(string homepage) Initializes the object with the homepage of the browser.
- void visit(string url) Visits url from the current page. It clears up all the forward history.
- string back(int steps) Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history at most steps.
- string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

Example:

```
BrowserHistory browserHistory = new BrowserHistory("leetcode.com");
browserHistory.visit("google.com");       // You are in "leetcode.com". Visit "google.com"
browserHistory.visit("facebook.com");     // You are in "google.com". Visit "facebook.com"
browserHistory.visit("youtube.com");      // You are in "facebook.com". Visit "youtube.com"
browserHistory.back(1);                   // You are in "youtube.com", move back to "facebook.com" return "facebook.com"
browserHistory.back(1);                   // You are in "facebook.com", move back to "google.com" return "google.com"
browserHistory.forward(1);                // You are in "google.com", move forward to "facebook.com" return "facebook.com"
browserHistory.visit("linkedin.com");     // You are in "facebook.com". Visit "linkedin.com"
browserHistory.forward(2);                // You are in "linkedin.com", you cannot move forward any steps.
browserHistory.back(2);                   // You are in "linkedin.com", move back two steps to "facebook.com" then to "google.com". return "google.com"
browserHistory.back(7);                   // You are in "google.com", you can move back only one step to "leetcode.com". return "leetcode.com"
```

Lab 07 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		

Data Structure Algorithms & Applications (CT-159)
Lab 08

Tree and Binary Tree

Objectives

The objective of the lab is to get students familiar with Binary Tree data structure, depth-first-traversal, breadth-first-traversal, insertion and deletion operations.

Tools Required

Dev C++ IDE

Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

Introduction

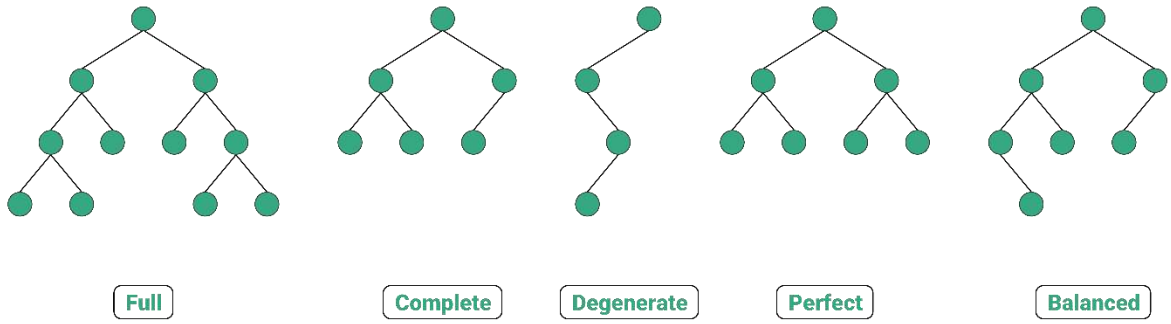
A **tree** is a hierarchical data structure consisting of nodes connected by edges. It is defined recursively as a collection of nodes, where the topmost node is called the **root**. Each node may have zero or more **child nodes**. Nodes with no children are called **leaf nodes**. Each child node has exactly one **parent** node, and there are no cycles, meaning a node cannot be its own ancestor.

Types of Trees: There are various types of trees based on different properties and applications. Below are the key types:

1. **General Tree**: No restrictions on the number of children per node. Used for representing hierarchical data structures like organizational charts, file systems, etc.
2. **Binary Tree**: Each node has at most two children, referred to as the **left** and **right** children. Binary trees are useful in representing expression trees, decision trees, etc.
3. **Complete Binary Tree**: All levels are completely filled except possibly the last level, which is filled from left to right. Used in heaps.
4. **Perfect Binary Tree**: A binary tree in which all interior nodes have exactly two children, and all leaves are at the same level.
5. **Balanced Tree**: A binary tree where the height is kept minimal, ensuring efficient operations (usually O(logn)).
6. **Full Binary Tree**: Every node other than the leaves has exactly two children.
7. **Binary Search Tree (BST)**: A binary tree where for every node, all nodes in the left subtree have smaller values, and all nodes in the right subtree have larger values. Allows for efficient searching, insertion, and deletion.
8. **Heap**: A complete binary tree used primarily in priority queues. **Max-Heap**: Parent nodes are always greater than or equal to their children. **Min-Heap**: Parent nodes are always less than or equal to their children.
9. **B-tree**: A balanced tree used in databases and file systems for efficient disk access. Nodes can have multiple children, and it maintains sorted data with efficient insertion, deletion, and search.
10. **Trie (Prefix Tree)**: A specialized tree used to store strings where each edge represents a character. Commonly used in autocomplete and spell-checking.

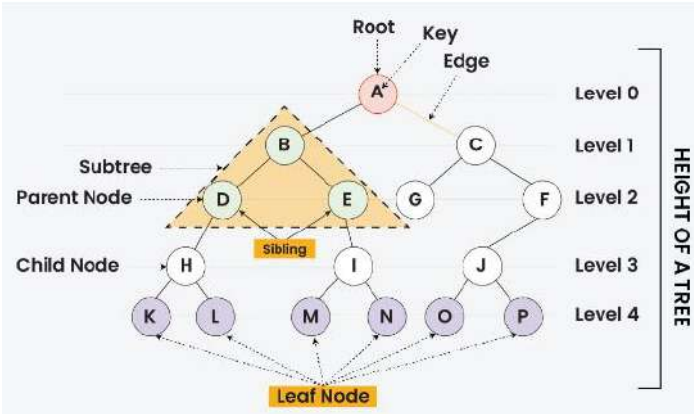
Binary Tree

A **binary tree** is a hierarchical data structure in which each node has at most two children. These children are referred to as the **left child** and the **right child**. The binary tree starts with a single node called the **root**.



Binary Tree Properties:

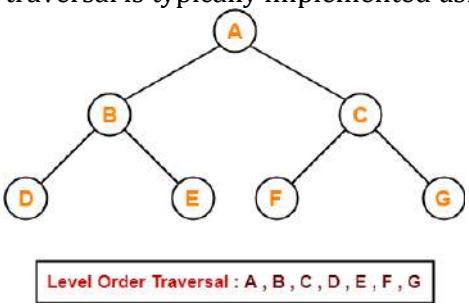
- **Depth of a Node**: The depth of a node is the number of edges from the root to the node. The root node has depth 0.
- **Degree of a Node**: The degree is the number of children of a node. The degree of a tree is the degree of the root node. In a binary tree, the degree of a node can be 0, 1 or 2.
- **Number of nodes at level l**: The level of a node is its depth. At level l, a binary tree can have at most 2^l nodes.
- **Maximum number of nodes in a binary tree of height h**: The maximum number of nodes is $2^h - 1$.
- **Minimum height of a binary tree with n nodes**: The minimum height is $\lceil log_2(n+1) \rceil$.



Binary Tree Traversals:

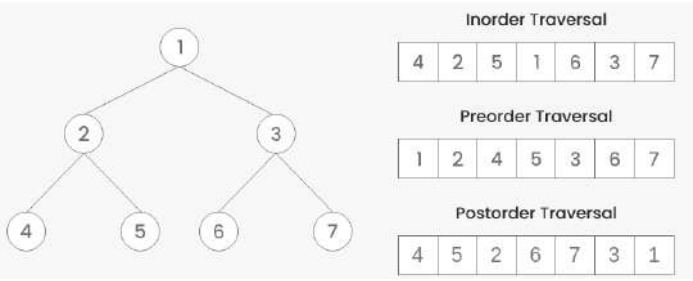
Binary tree traversal refers to the process of visiting each node in a binary tree in a specific order. Traversals are essential for many operations, such as searching, printing, or modifying tree nodes. There are two primary categories of binary tree traversals: **depth-first traversal** and **breadth-first traversal**.

In **breadth-first traversal,** the tree is explored level by level, starting from the root. Nodes at each level are visited from left to right. This traversal is useful for searching the shortest path in an unweighted tree. Breadth-first traversal is typically implemented using a **queue** data structure.



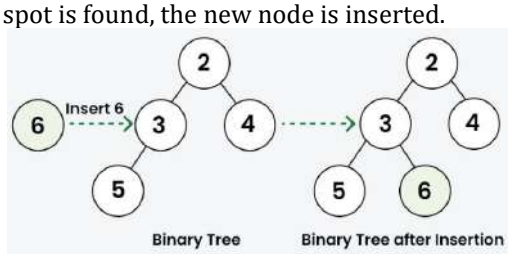
In **depth-first traversal**, the algorithm explores as far down one branch as possible before backtracking. There are three types of depth-first traversals:

1. Inorder Traversal (Left, Root, Right): Traverses the left subtree, visits the root, and then traverses the right subtree.
2. Preorder Traversal (Root, Left, Right): Visits the root first, then traverses the left subtree, and finally the right subtree.
3. Postorder Traversal (Left, Right, Root): Traverses the left subtree, then the right subtree, and finally visits the root.



Insert a Node in Binary Tree

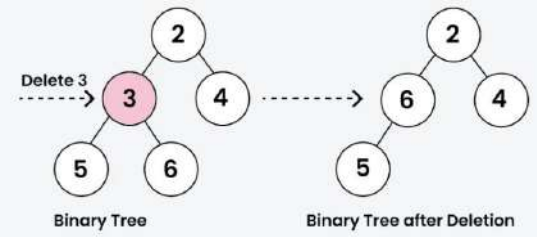
1. If the binary tree is empty, the new node becomes the root.
2. Otherwise, a level-order traversal (using a queue) is performed to find the first empty spot (either in the left or right child).
3. Once the first available spot is found, the new node is inserted.



This approach ensures that the new node is added in a complete binary tree fashion, filling levels from left to right.

Delete a Node from a Binary Tree

1. **Finding the node to delete**: We first perform a level-order traversal to find the node that needs to be deleted (nodeToDelete) and keep track of the deepest node in the tree (deepestNode).
2. **Replacement**: Once the nodeToDelete is found, we replace its value with the value of the deepestNode.
3. **Deleting the deepest node**: We then delete the deepest node to ensure that the binary tree structure remains valid.
4. **Edge case**: If the tree is empty or only has one node, handle that accordingly.



This approach keeps the binary tree complete while ensuring that the node is properly deleted.

Example 01: Implementation of a binary tree with traversal, search, insert and delete operations.

```
#include <iostream>
#include <queue>
using namespace std;

class Node {   // Node structure for the binary tree
   public:
   int data;
   Node* left;
   Node* right;

   Node(int value) {    // Constructor to initialize node
      data = value;
      left = nullptr;
      right = nullptr;
   }
};
class BinaryTree {    // Binary Tree Class
private:
   Node* root;

public:
   BinaryTree() {
      root = nullptr;
   }

   void insert(int data) {     // Function to insert data in level-order (simple binary tree, not based on value comparison)
      Node* newNode = new Node(data);
      if (root == nullptr) {       // If the tree is empty, set the new node as the root
         root = newNode;
         return;
      }
      queue<Node*> q;             // Use a queue for level-order traversal to find the first available position
      q.push(root);
      while (!q.empty()) {
         Node* current = q.front();
         q.pop();

         // Check the left child
         if (current->left == nullptr) {
            current->left = newNode;
            return;
         } else {
            q.push(current->left);
         }

         // Check the right child
         if (current->right == nullptr) {
            current->right = newNode;
            return;
         } else {
```

```
            q.push(current->right);
        }
    }
  }

  void inorder() {   // Function to perform inorder traversal (left -> root -> right)
    inorderTraversal(root);
    cout << endl;
  }
private:
  void inorderTraversal(Node* node) {          // Recursive inorder traversal function
    if (node == nullptr) return;
    inorderTraversal(node->left);
    cout << node->data << " ";
    inorderTraversal(node->right);
  }
};

int main() {
  BinaryTree tree;
  // Insert values into the binary tree in level-order
  tree.insert(1);
  tree.insert(2);s
  tree.insert(3);
  tree.insert(4);
  tree.insert(5);
  tree.insert(6);
  tree.insert(7);

  cout << "Inorder Traversal: ";
  tree.inorder();
}
```

Key Use Cases for Different Traversals:

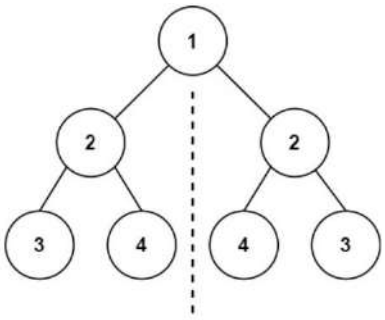
- Inorder Traversal: Used in binary search trees to retrieve nodes in sorted order.
- Preorder Traversal: Used to copy the tree or to prefix-express an arithmetic expression stored in a tree.
- Postorder Traversal: Useful for deleting or freeing nodes in a tree or evaluating postfix arithmetic expressions.
- Level-Order Traversal: Used in breadth-first search and is ideal for finding the shortest path in unweighted trees or graphs.

Applications of Binary Trees:

- Expression Trees: Used to represent arithmetic expressions.
- Binary Search Trees: Used for searching, inserting, and deleting data efficiently.
- Heap Data Structure: A special binary tree used for implementing priority queues.
- Huffman Coding Tree: Used in data compression algorithms.

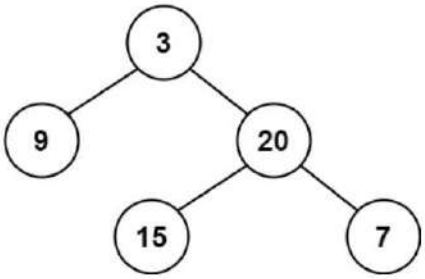
Exercise

1. Implement methods for the following operations in Binary tree class given in example 01.
    - Searching a node based on a given value.
    - Pre-order traversal
    - Post-order traversal
2. Given the root of a binary tree, *check whether it is a mirror of itself* (i.e., symmetric around its center).

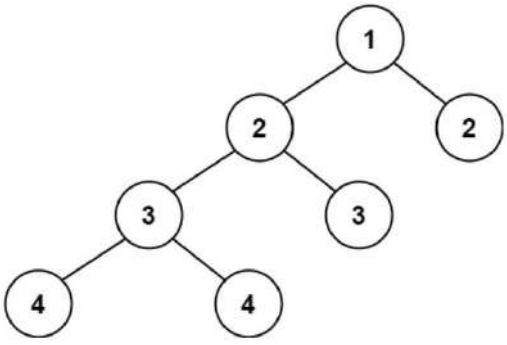



```
Input: root = [1,2,2,3,4,4,3]
Output: true
```

3. Given a binary tree, determine if it is height-balanced i.e., the absolute difference between the left and right subtree of each node is not greater than 1.

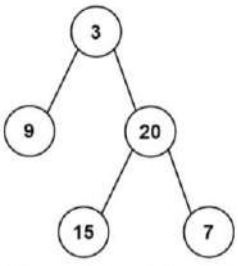



```
Input: root = [3,9,20,null,null,15,7]
Output: true
```



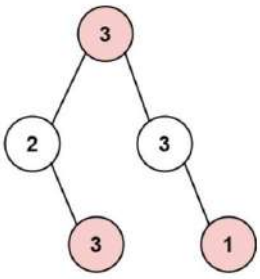

```
Input: root = [1,2,2,3,3,null,null,4,4]
Output: false
```

4. Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

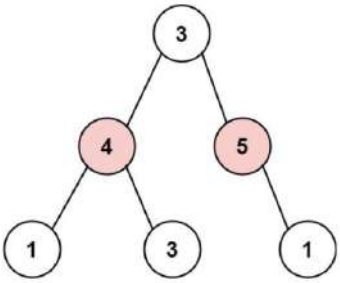



```
Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
Output: [3,9,20,null,null,15,7]
```

5. The thief has found himself a new place for his thievery again. There is only one entrance to this area, called root. Besides the root, each house has one and only one parent house. After a tour, the smart thief realized that all houses in this place form a binary tree. It will automatically contact the police if two directly-linked houses were broken into on the same night. Given the root of the binary tree, return the maximum amount of money the thief can rob without alerting the police.



Input: root = [3,2,3,null,3,null,1]
Output: 7
Explanation: Maximum amount of money the thief can rob = 3 + 3 + 1 = 7.



Input: root = [3,4,5,1,3,null,1]
Output: 9
Explanation: Maximum amount of money the thief can rob = 4 + 5 = 9.

Lab 08 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		

Data Structure Algorithms & Applications (CT-159)
Lab 09

Binary Search Tree

Objectives

The objective of the lab is to get students familiar with Binary Search Tree and its operations i.e., insert, remove, search, traversal.

Tools Required

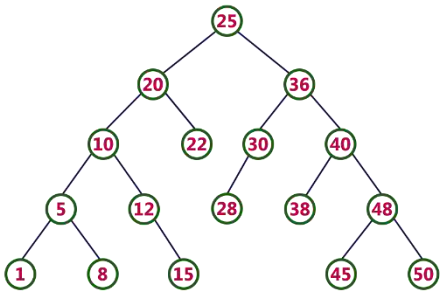
Dev C++ IDE

Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

Binary Search Tree

A **Binary Search Tree (BST)** is a type of binary tree where each node has at most two children, commonly referred to as the left child and the right child. For each node:

- **Left Subtree**: All the nodes in the left subtree have values less than the node's value.
- **Right Subtree**: All the nodes in the right subtree have values greater than the node's value.
- Both left and right subtrees must also be binary search trees.

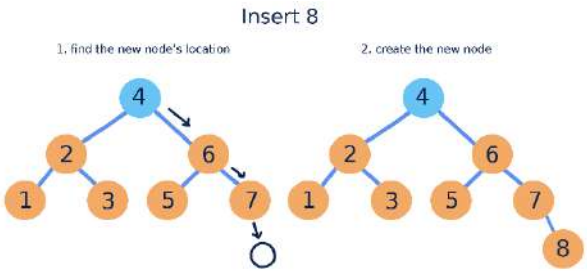


Properties:

- **Binary Tree Structure**: A BST is a special case of a binary tree where the order property is maintained.
- **In-Order Traversal**: In-order traversal of a BST yields the values in **sorted (ascending) order**.
- **Time Complexity**: **Search, Insert, and Delete**: On average, these operations are O(logn), where n is the number of nodes. However, in the worst case (if the tree becomes unbalanced), the time complexity is O(n).
- **Height**: The height of a balanced BST is O(logn), but it can degrade to O(n) in a degenerate case (if the tree becomes skewed).

Insert a Node in a BST:

If root is nullptr: Create a new node with value
If value < root.data: Set root.left to insertNode(root.left, value) (recursively insert in the left subtree)
Else If value > root.data: Set root.right to insertNode(root.right, value) (recursively insert in the right subtree)



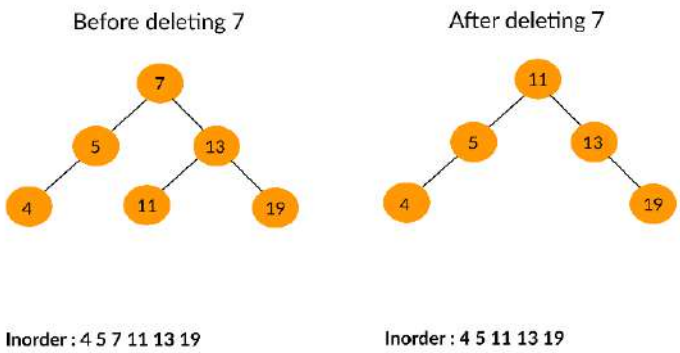
Delete a Node from a BST:

Find the Node: Start from the root and traverse the tree based on the value you want to delete.

- Move left if the value is smaller.
- Move right if the value is larger.

Delete the Node:

- **Case 1**: Node has **no children**: Simply remove the node.
- **Case 2**: Node has **one child**: Replace the node with its only child.
- **Case 3**: Node has **two children**: Find the **inorder successor** (smallest value in the right subtree), copy its value to the node to be deleted, and recursively delete the inorder successor.



Example 01: Implementation of a binary search tree.

```
#include <iostream>
using namespace std;
class TreeNode {
public:
   int val;
   TreeNode* left;
   TreeNode* right;
   TreeNode(int value) : val(value), left(NULL), right(NULL) {}
};

class BinarySearchTree {
public:
   TreeNode* root;
   BinarySearchTree() : root(NULL) {}

   void insert(int key) {
      root = insertRec(root, key);
   }
   void deleteNode(int key) {
      root = deleteRec(root, key);
   }

   void inorder() {
      inorderRec(root);
      cout << endl;
   }

private:
   // Recursive function to insert a node
   TreeNode* insertRec(TreeNode* node, int key) {
      // Base case: if the tree is empty, create a new node
      if (node == NULL)
         return new TreeNode(key);

      // Otherwise, recur down the tree
      if (key < node->val)
         node->left = insertRec(node->left, key);
      else if (key > node->val)
         node->right = insertRec(node->right, key);

      return node;
   }

   // Recursive function to delete a node
   TreeNode* deleteRec(TreeNode* node, int key) {
      // Base case: if the tree is empty
      if (node == NULL)
         return node;

      // Recur down the tree
      if (key < node->val)
         node->left = deleteRec(node->left, key);
      else if (key > node->val)
         node->right = deleteRec(node->right, key);
      else {
         // Node with only one child or no child
         if (node->left == NULL) {
            TreeNode* temp = node->right;
            delete node;
            return temp;
         } else if (node->right == NULL) {
            TreeNode* temp = node->left;
            delete node;
            return temp;
         }

         // Node with two children: Get the inorder successor (smallest in the right subtree)
         TreeNode* temp = minValueNode(node->right);
         // Copy the inorder successor's content to this node
         node->val = temp->val;
```

```
        // Delete the inorder successor
        node->right = deleteRec(node->right, temp->val);
    }
    return node;
  }
  // Find the node with the minimum value (used for deleting nodes)
  TreeNode* minValueNode(TreeNode* node) {
    TreeNode* current = node;
    while (current && current->left != NULL)
      current = current->left;
    return current;
  }
  // Inorder traversal function
  void inorderRec(TreeNode* root) {
    if (root != NULL) {
      inorderRec(root->left);
      cout << root->val << " ";
      inorderRec(root->right);
    }
  }
};
int main() {
  BinarySearchTree bst;
  bst.insert(50);
  bst.insert(30);
  bst.insert(20);
  bst.insert(40);
  bst.insert(70);
  bst.insert(60);
  bst.insert(80);
  cout << "Inorder traversal of the tree: ";
  bst.inorder();
  cout << "Delete 20\n";
  bst.deleteNode(20);
  cout << "Inorder traversal of the tree after deleting 20: ";
  bst.inorder();
}
```

Issues with BST

- **Unbalanced Tree**: In the worst case (e.g., inserting sorted data), the tree can become unbalanced, degrading to a linked list, which results in O(n) time complexity for operations.
- **Rebalancing Required**: To maintain efficiency, balancing techniques such as AVL or Red-Black Trees are often used, but these come with added complexity.
- **Memory Usage**: BSTs use more memory than arrays or linked lists due to the storage overhead for pointers (two for each node).

Computational Complexity Comparison of Data Structures covered so far

Data Structure	Insertion	Deletion	Search	Access (Indexing)
Linked List	O(1) (at head/tail)	O(n) (unsorted)	O(n)	O(n)
Array	O(n) (due to shifting)	O(n) (due to shifting)	O(n) (unsorted)	O(1)
Binary Tree	O(n)	O(n)	O(n)	N/A (no direct indexing)
Binary Search Tree	O(h) = O(log n) (balanced), O(n) (unbalanced)	O(h) = O(log n) (balanced), O(n) (unbalanced)	O(h) = O(log n) (balanced), O(n) (unbalanced)	N/A (no direct indexing)

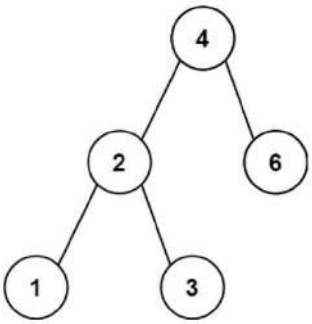
Exercise

1. Define following methods in Example 01 by using stack.
    - Preorder traversal
    - Postorder travsersal

2. You are tasked with designing an employee management system for a small company. Each employee has a unique ID, name, and department. You are required to store and manage employee records in a way that allows quick insertion, deletion, and search operations based on the employee ID. You decide to use a Binary Search Tree (BST) to store the employee records. Each node in the BST will store: Employee ID (used as the key for the BST), Employee Name, Employee Department.

- Create a C++ class EmployeeNode representing each employee in the BST. Each node should store: int id (Employee ID), string name (Employee Name), string department (Employee Department), A pointer to the left child (EmployeeNode* left), a pointer to the right child (EmployeeNode* right).
- Create a class EmployeeBST with the following member functions:
    - insert(int id, string name, string department): Inserts a new employee into the BST.
    - search(int id): Searches for an employee by ID. If found, return their name and department; otherwise, print an appropriate message.
    - deleteNode(int id): Deletes an employee from the BST based on their ID.
    - inOrderTraversal(): Prints all employees in ascending order of their ID, showing their ID, name, and department.
    - findMin(): Returns the name and department of the employee with the smallest ID.
    - findMax(): Returns the name and department of the employee with the largest ID.
- Also, Handle edge cases such as: Inserting an employee with a duplicate ID, Deleting an employee that does not exist, Deleting nodes with no children, one child, and two children, Handling an empty tree for search or delete operations.

3. Given the two nodes of a binary search tree, return their least common ancestor.
4. Given the root of a binary search tree, recursively find the sum of all nodes of the tree.
5. Given the root of a Binary Search Tree (BST), return the minimum difference between the values of any two different nodes in the tree.




```
Input: root = [4,2,6,1,3]
Output: 1
```

Lab 09 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		

Data Structure Algorithms & Applications (CT-159)
Lab 10

AVL Tree

Objectives

The objective of the lab is to get students familiar with AVL Tree and rotations.

Tools Required

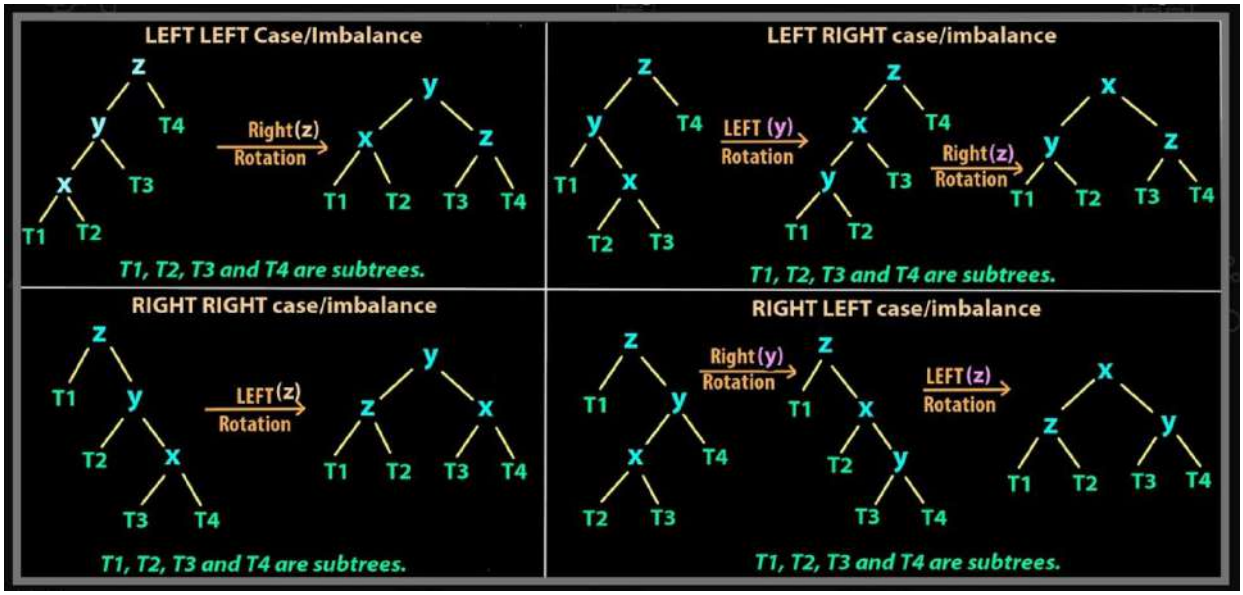
Dev C++ IDE

Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

AVL Tree

AVL trees (named after their inventors Adelson-Velsky and Landis) are **self-balancing binary search trees** that maintain a balance factor for each node to ensure the tree remains balanced. They provide guaranteed O(log n) time complexity for search, insertion, and deletion operations. Here's how they solve the problems in BSTs:

- **Height-Balanced Property**: In an AVL tree, the difference in the height of the left and right subtrees for any node (called the **balance factor**) is either -1, 0, or 1. Balance factor = Height of the left subtree – Height of the right subtree. Whenever an insertion or deletion operation leads to an imbalance, **rotations** (left, right, or a combination) are performed to restore balance.
- **Self-Balancing**: After every insertion or deletion, the tree checks if it is still balanced by calculating the balance factor for each node. If the balance factor is violated, the tree performs **rotations** to re-balance itself. This prevents the tree from becoming too skewed and ensures that the height of the tree is kept at O(log n).
- **Rotations**: AVL trees use **four types of rotations** to maintain balance: **Right Rotation (Single)**: Performed when a node becomes unbalanced due to too much height on its left subtree (left-left case). **Left Rotation (Single)**: Performed when a node becomes unbalanced due to too much height on its right subtree (right-right case). **Left-Right Rotation (Double)**: Performed when a node's left child is right-heavy (left-right case). **Right-Left Rotation (Double)**: Performed when a node's right child is left-heavy (right-left case). These rotations effectively reduce the height of the tree and restore balance.



Steps for Inserting a Node in an AVL tree

1. Perform standard BST insertion.
2. Update the height of each node.
3. Calculate the balance factor for each node.
4. If unbalanced, apply the appropriate rotation(s).

Steps for Deleting a Node from an AVL tree

1. Perform Standard BST Deletion.
2. Update the height for each node.
3. Check Balance Factor for each node.
4. Perform Rotations (if needed). If the tree is unbalanced, apply one of the four rotations: LL, RR, LR, or RL to restore balance.
5. Repeat Until the Tree is Fully Balanced. Ensure that every node along the path from the deleted node up to the root is balanced. Multiple rotations might be needed.

Example 01: Implementation of an AVL tree.

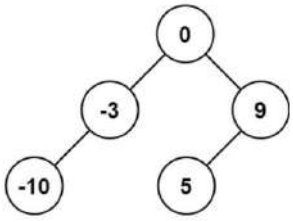
```
#include <iostream>
using namespace std;

class AVLTree {
private:
   struct Node {
      int key;
      Node* left;
      Node* right;
      int height;

      Node(int k) : key(k), left(nullptr), right(nullptr), height(1) {}
   };

   Node* root;
```

```
int height(Node* node) {
   return node ? node->height : 0;
}

int balanceFactor(Node* node) {
   return node ? height(node->left) - height(node->right) : 0;
}

void updateHeight(Node* node) {
   node->height = 1 + max(height(node->left), height(node->right));
}

Node* rotateRight(Node* y) {
   Node* x = y->left;
   Node* T2 = x->right;

   x->right = y;
   y->left = T2;

   updateHeight(y);
   updateHeight(x);

   return x;
}

Node* rotateLeft(Node* x) {
   Node* y = x->right;
   Node* T2 = y->left;

   y->left = x;
   x->right = T2;

   updateHeight(x);
   updateHeight(y);

   return y;
}

Node* balance(Node* node) {
   updateHeight(node);

   int balance = balanceFactor(node);

   // Left heavy
   if (balance > 1) {
      if (balanceFactor(node->left) < 0) // Left-Right case
         node->left = rotateLeft(node->left);
      return rotateRight(node); // Left-Left case
   }

   // Right heavy
   if (balance < -1) {
      if (balanceFactor(node->right) > 0) // Right-Left case
         node->right = rotateRight(node->right);
      return rotateLeft(node); // Right-Right case
   }

   return node; // Balanced
}

Node* minValueNode(Node* node) {
   Node* current = node;
   while (current && current->left != nullptr)
      current = current->left;
   return current;
}

Node* insert(Node* node, int key) {
```

```
        //recursive insertion in a bst
        // implement the code

        return balance(node);
    }

    Node* deleteNode(Node* root, int key) {
         // Perform standard BST delete
        // Complete the code

        // Balance the node
        return balance(root);
    }

    void inOrder(Node* node) {
        if (!node) return;
        inOrder(node->left);
        cout << node->key << " ";
        inOrder(node->right);
    }

public:
    AVLTree() : root(nullptr) {}

    void insert(int key) {
        root = insert(root, key);
    }

    void deleteKey(int key) {
        root = deleteNode(root, key);
    }

    void inOrder() {
        inOrder(root);
        cout << endl;
    }
};

int main() {
    AVLTree tree;
    tree.insert(10);
    tree.insert(20);
    tree.insert(30);
    tree.insert(20);  // Duplicate insertion
    tree.insert(40);
    tree.insert(50);
    tree.insert(25);

    cout << "In-order traversal before deletion: ";
    tree.inOrder();

    tree.deleteKey(20);
    cout << "In-order traversal after deleting 20: ";
    tree.inOrder();

    return 0;
}
```

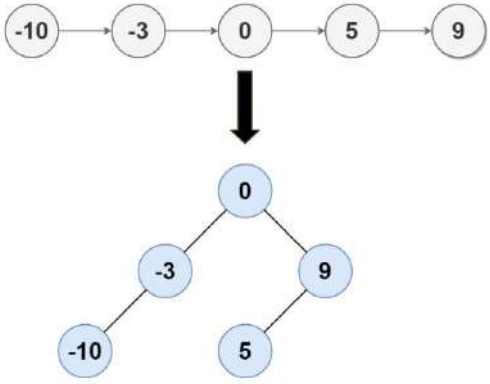
Exercise

1. Complete implementation of AVL tree given in Example 01 by implementing Insert and Delete methods.
2. Given an integer array nums where the elements are sorted in ascending order, convert it to a height-balanced binary search tree.




```
Input: nums = [-10,-3,0,5,9]
Output: [0,-3,9,-10,null,5]
Explanation: [0,-10,5,null,-3,null,9] is also accepted:
```

3. Given the head of a singly linked list where elements are sorted in ascending order, convert it to a height-balanced binary search tree.




```
Input: head = [-10,-3,0,5,9]
Output: [0,-3,9,-10,null,5]
Explanation: One possible answer is [0,-3,9,-10,null,5], which represents the
shown height balanced BST.
```

4. Given the root of a BST, find its diameter.

Lab 10 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		

Data Structure Algorithms & Applications (CT-159)
Lab 11

Heap and Priority Queue

Objectives

The objective of the lab is to get students familiar with Heap and Priority Queue Data Structures.

Tools Required

Dev C++ IDE

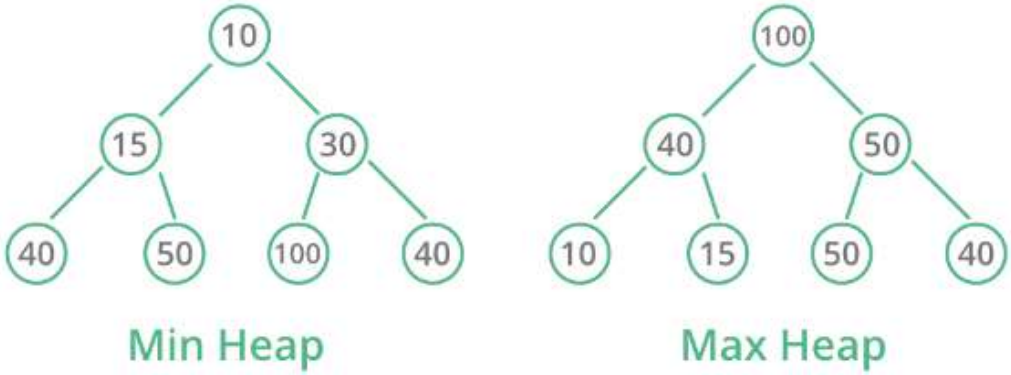
Course Coordinator –
Course Instructor –
Lab Instructor –
Prepared By Department of Computer Science and Information Technology
NED University of Engineering and Technology

Heap

A **heap** is a binary tree-based **data structure** that follows the **heap property**. In a heap, the value of each node is compared to the values of its children in a specific way:

- **Max-Heap**: The value of each node is greater than or equal to the values of its children, ensuring that the root node contains the maximum value. As you move down the tree, the values decrease.
- **Min-Heap**: The value of each node is less than or equal to the values of its children, ensuring that the root node contains the minimum value. As you move down the tree, the values increase.

This property guarantees that the largest (in a max-heap) or smallest (in a min-heap) value is always at the root, and the tree maintains a specific order based on the type of heap. Heaps are usually used to implement priority queues, efficient sorting (heapsort), and finding the k-th largest or smallest element.



Heap Implementation:

A heap can be implemented using a dynamic array. The root of the heap is at index 0. For each element at index i, its:

- Parent is at index (i-1)/2
- Left child is at index 2*i + 1
- Right child is at index 2*i + 2

Heapify:

It is the process to rearrange the elements to maintain the property of heap data structure. It is done when a certain node creates an imbalance in the heap due to some operations like insertion and deletion. It takes **O(log N)** to balance the tree. **Heapify up** is used after inserting a new element at the end of the heap to restore the heap property by moving the element up to its correct position. **Heapify down** is used after removing the root (max/min element) from the heap. The last element is moved to the root, and heapify down restores the heap property by moving the element down to its correct position.

Insertion: If we insert a new element into the heap since we are adding a new element into the heap so it will distort the properties of the heap so we need to perform the **heapify-up** operation so that it maintains the property of the heap.

Heapify Up Operation:

Insert the new element at the last position in the heap (end of the array).
Set i as the index of the newly inserted element.
While i > 0 and heap[parent(i)] < heap[i] (for max-heap):

- Swap heap[i] with heap[parent(i)].
- Set i to parent(i).

Stop when the heap property is restored

```
void heapifyUp(int i) {
  while (i > 0 && heap[(i - 1) / 2] < heap[i]) {
    swap(heap[i], heap[(i - 1) / 2]);
    i = (i - 1) / 2; // Move to the parent
  }
}
```

Deletion: If we delete the element from the heap it always deletes the root element of the tree and replaces it with the last element of the tree. Since we delete the root element from the heap it will distort the properties of the heap so we need to perform heapify down operations so that it maintains the property of the heap.

Heapify Down Operation:
Replace heap[0] (root) with the last element and remove the last element.
Set i as the index of the new root.
While i has a left child (2 * i + 1 < heap_size):

- Let largest be the index of the left child.
- If i has a right child (2 * i + 2 < heap_size) and the right child is greater than the left child, set largest to the right child.
- If heap[i] is smaller than heap[largest], swap them.
- Set i to largest.

Stop when the heap property is restored

```
void heapifyDown(int i) {
   int size = heap.size();
   while (2 * i + 1 < size) { // While there's a left child
      int largest = 2 * i + 1; // Assume left child is largest
      int right = 2 * i + 2;   // Right child index
      if (right < size && heap[right] > heap[largest]) {
         largest = right; // Right child is larger
      }
      if (heap[i] >= heap[largest]) {
         break; // Heap property is satisfied
      }
      swap(heap[i], heap[largest]); // Swap with larger child
      i = largest; // Move to the larger child
   }
}
```

Example 01: Implementation of a Max Heap Class

```
#include <iostream>
#include <vector>
using namespace std;
class MaxHeap {
private:
   vector<int> heap;

   // Helper method to heapify up (for insertion)
   void heapifyUp(int index) {
      while (index > 0 && heap[parent(index)] < heap[index]) {
         swap(heap[parent(index)], heap[index]);
         index = parent(index);
      }
   }
   // Helper method to heapify down (for deletion)
   void heapifyDown(int index) {
      int size = heap.size();
      while (leftChild(index) < size) { // while there is a left child
         int largest = index;
         int left = leftChild(index);
         int right = rightChild(index);

         if (heap[left] > heap[largest]) largest = left;
         if (right < size && heap[right] > heap[largest]) largest = right;

         if (largest != index) {
            swap(heap[index], heap[largest]);
            index = largest;
         } else {
            break;  // heap property satisfied
         }
      }
   }
```

```
    // Helper functions to get parent and children indices
    int parent(int i) { return (i - 1) / 2; }
    int leftChild(int i) { return 2 * i + 1; }
    int rightChild(int i) { return 2 * i + 2; }
public:
    // Insert a new value into the heap
    void insert(int value) {
        heap.push_back(value); // Add value at the end
        heapifyUp(heap.size() - 1); // Restore heap property
    }
    // Remove the maximum element (the root)
    void extractMax() {
        if (heap.empty()) {
            throw out_of_range("Heap is empty");
        }
        int maxValue = heap[0];
        heap[0] = heap.back();  // Move last element to root
        heap.pop_back();  // Remove the last element
        heapifyDown(0);  // Restore heap property
    }
    // Get the maximum element without removing it
    int getMax() const {
        if (heap.empty()) {
            throw out_of_range("Heap is empty");
        }
        return heap[0];
    }
    // Check if the heap is empty
    bool isEmpty() const {
        return heap.empty();
    }
};

// Main function to demonstrate the MaxHeap class
int main() {
    MaxHeap heap;
    heap.insert(20);
    heap.insert(15);
    heap.insert(30);
    heap.insert(40);
    heap.insert(10);
    cout << "Max element: " << heap.getMax() << endl;
    cout << "Extracting max element: " << heap.extractMax() << endl;
    cout << "Max-element after extraction: ";
    cout << "Max element: " << heap.getMax() << endl;
}
```

Heap Sort

Heap Sort is an efficient comparison-based sorting algorithm that uses the **heap data structure** to sort an array. The key idea is to first build a max-heap from the input array and then repeatedly extract the maximum element (root of the heap), placing it at the end of the array. The process is repeated until the array is sorted.

Steps for Heap Sort:

1. **Build a Max-Heap** from a given unsorted input array. Use from-bottom-to-top approach.
2. **Extract the maximum element** (the root) from the heap, and move it to the end of the array.
3. **Heapify the root** to maintain the heap property for the reduced heap (ignoring the last element which is now sorted).
4. Repeat the process until all elements are extracted and placed in their correct position.

Priority Queue

A **priority queue** is a data structure that allows efficient retrieval of the "highest priority" element. The most common implementation of a priority queue is using a **heap** because heaps provide efficient insertion and deletion (retrieval of the highest or lowest element, depending on whether it's a max-heap or min-heap).

Time Complexity:

- **Insertion**: O(log n) — Heapify up takes logarithmic time in the worst case.
- **Deletion (pop)**: O(log n) — Heapify down takes logarithmic time.

- **Top (peek)**: O(1) — Simply accessing the root element.
- **Space Complexity**: O(n) — Space to store the elements in the heap.

Priority Queue Library

In C++, the Standard Template Library (STL) provides a priority_queue container, which is essentially a max-heap by default. This means the largest element is always at the top. The priority_queue container is part of the <queue> header.

Key Features of priority_queue:

- **Automatic ordering**: It automatically orders elements such that the largest (by default) or smallest (if using a custom comparator) element is accessible at the top.
- **Custom ordering**: By providing a custom comparator, you can turn it into a min-heap or create custom priority rules.
- **Operations**:
  - Insertion (push)
  - Access to the top element (top)
  - Removal of the top element (pop)
  - Check size (size), empty state (empty)

Example 02: Using Priority queue library as a max-heap.

```
#include <iostream>
#include <queue>
using namespace std;
int main() {
   priority_queue<int> pq;
   // Insert elements
   pq.push(10);
   pq.push(30);
   pq.push(20);
   pq.push(5);
   // Display the top element (max element)
   cout << "Top element: " << pq.top() << endl; // 30
   // Remove elements (pops the largest element)
   pq.pop();
   cout << "Top element after pop: " << pq.top() << endl; // 20
   return 0;
}
```

Example 02: Using Priority queue library as a min-heap.

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// Custom comparator for a max-heap
//functor : a class that overloads function operator (). it invokes function whenever its instance/object is created
class CustomCompare {
   public:
    bool operator()(int a, int b) {
      return a < b;  // Max-heap: larger elements should have higher priority
   }
};
int main() {
   // Max-heap using a custom comparator and vector as the container
   priority_queue<int, vector<int>, CustomCompare> pq;
   pq.push(10);
   pq.push(30);
   pq.push(20);
   pq.push(5);
   while (!pq.empty()) {
      cout << pq.top() << " ";  // Output: 30 20 10 5
      pq.pop();
   }
   return 0;
}
```

Exercise

1. Implement Heap Sort by using the steps listed in the Lab manual on Page#05.
2. Given an integer array nums and an integer k, return the k most frequent elements. You may return the answer in any order.
   Example 1: Input: nums = [1,1,1,2,2,3], k = 2, Output: [1,2]
   Example 2: Input: nums = [1], k = 1, Output: [1]
3. Given a string s, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string. Return the sorted string. If there are multiple answers, return any of them.
   Example 1: Input: s = "tree", Output: "eert"
   Example 2: Input: s = "cccaaa", Output: "aaaccc"
4. There are n workers. You are given two integer arrays quality and wage where quality[i] is the quality of the ith worker and wage[i] is the minimum wage expectation for the ith worker. We want to hire exactly k workers to form a paid group. To hire a group of k workers, we must pay them according to the following rules: Every worker in the paid group must be paid at least their minimum wage expectation. In the group, each worker's pay must be directly proportional to their quality. This means if a worker's quality is double that of another worker in the group, then they must be paid twice as much as the other worker. Given the integer k, return the least amount of money needed to form a paid group satisfying the above conditions. Answers within 10^-5 of the actual answer will be accepted.
   Example 1: Input: quality = [10,20,5], wage = [70,50,30], k = 2, Output: 105.00000
   Explanation: We pay 70 to 0th worker and 35 to 2nd worker.
   Example 2: Input: quality = [3,1,10,10,1], wage = [4,8,2,2,7], k = 3, Output: 30.66667
   Explanation: We pay 4 to 0th worker, 13.33333 to 2nd and 3rd workers separately.
5. The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value. So the median is the mean of the two middle values. For examples, if arr = [2,3,4], the median is 3. For examples, if arr = [1,2,3,4], the median is (2 + 3) / 2 = 2.5. You are given an integer array nums and an integer k. There is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position. Return the median array for each window in the original array. Answers within 10^-5 of the actual value will be accepted.
   Example 1: Input: nums = [1,3,-1,-3,5,3,6,7], k = 3,
   Output: [1.00000,-1.00000,-1.00000,3.00000,5.00000,6.00000]

Lab 11 Evaluation		
Student Name:	**Student ID:**	**Date:**
Rubric	**Marks (25)**	**Remarks by teacher in accordance with the rubrics**
R1		
R2		
R3		
R4		
R5		